Yogesh Simmhan
simmhan@iisc.ac.in

Department of Computational and Data Sciences
Indian Institute of Science, Bangalore

DSBA DA-2310 (3:1)

Data Engineering at Scale

Upcoming Assignments

- Paper Presentations *(10mins each)*
  - **Tue 8 April, 2025, 430-530PM *(6 slots)***
  - **Thu 10 April, 2025, 330-530PM *(10 slots)***
  - **Fri 11 April, 2025, 10AM-12PM *(10 slots)***
  - ***https://forms.office.com/r/KDNVAycYf8***
- Assignment 1 Deadline
  - **13 April, 2025**
  - *Update to be posted today. Complete assignment this week. Use next week for experiments, reports.*
- Final Exam
  - **Tue 29 April, 3-5PM**

Project report *(One per team. Do NOT submit duplicates!)*

- 6-8 A4 pages, 10 point font, 2.5cm margins, PDF
- Key sections:
  - Heading: Title, Authors and Email Address
  - Problem: Definition , Motivation, Design Goals, Features required, Scalability/Performance Goals
  - Approach and Methods: High level design, Architecture/Data Model, Big Data platforms used, ML Methods used
  - Evaluation: Experiment design, Scalability/Performance metrics, Feature metrics, Plots and Analysis
  - Summary: Ability to achieve design and performance goal wrt Proposal, Future extensions
- Code to be uploaded to Git
- Demo and Presentation
- Report Submission by **May 1**
- Demo and Presentation on **Sat, May 3 from 930AM-230PM**
  - *Hybrid for those who are traveling*

Upcoming Lectures

- 1/April: Kafka, Spark Streaming
- 3/April: Pregel/Giraph for graph processing
- 8/April (330-430): Pregel/Giraph conclusion

Mandatory Reading

- Kafka: A Distributed Messaging System for Log Processing, Jay Kreps, Neha Narkhede, Jun Rao, NetDB, 2011
- Discretized Streams: Fault-Tolerant Streaming Computation at Scale, Matei Zaharia, et al, SOSP 2013
- Pregel: a system for large-scale graph processing, Malewicz, et al, ACM SIGMOD 2010

Module 5

Linked Data & Fast Data Processing

Motivation

The FOUR V's of Big Data

From traffic patterns and music downloads to web history and medical records, data is recorded, stored, and analyzed to enable the technology and services that the world relies on every day. But what exactly is big data, and how can these massive amounts of data be used?

As a leader in the sector, IBM data scientists break big data into four dimensions: **Volume, Velocity, Variety and Veracity**

Depending on the industry and organization, big data encompasses information from multiple internal and external sources such as transactions, social media, enterprise content, sensors and mobile devices. Companies can leverage data to adapt their products and services to better meet customer needs, optimize operations and infrastructure, and find new sources of revenue.

By 2015
4.4 MILLION IT JOBS
will be created globally to support big data, with 1.9 million in the United States

Volume
SCALE OF DATA

40 ZETTABYTES
[43 TRILLION GIGABYTES]
of data will be created by 2020, an increase of 300 times from 2005

It's estimated that
2.5 QUINTILLION BYTES
[2.3 TRILLION GIGABYTES]
of data are created each day

6 BILLION PEOPLE
have cell phones

WORLD POPULATION: 7 BILLION

Most companies in the U.S. have at least
100 TERABYTES
[100,000 GIGABYTES]
of data stored

Variety
DIFFERENT FORMS OF DATA

As of 2011, the global size of data in healthcare was estimated to be
150 EXABYTES
[161 BILLION GIGABYTES]

By 2014, it's anticipated there will be
420 MILLION WEARABLE, WIRELESS HEALTH MONITORS

4 BILLION+ HOURS OF VIDEO
are watched on YouTube each month

30 BILLION PIECES OF CONTENT
are shared on Facebook every month

400 MILLION TWEETS
are sent per day by about 200 million monthly active users

Velocity
ANALYSIS OF STREAMING DATA

The New York Stock Exchange captures
1 TB OF TRADE INFORMATION
during each trading session

Modern cars have close to
100 SENSORS
that monitor items such as fuel level and tire pressure

By 2016, it is projected there will be
18.9 BILLION NETWORK CONNECTIONS
– almost 2.5 connections per person on earth

Veracity
UNCERTAINTY OF DATA

1 IN 3 BUSINESS LEADERS
don't trust the information they use to make decisions

in one survey were unsure of how much of their data was inaccurate

Poor data quality costs the US economy around
$3.1 TRILLION A YEAR

Sources: McKinsey Global Institute, Twitter, Cisco, Gartner, EMC, SAS, IBM, MEPTEC, QAS

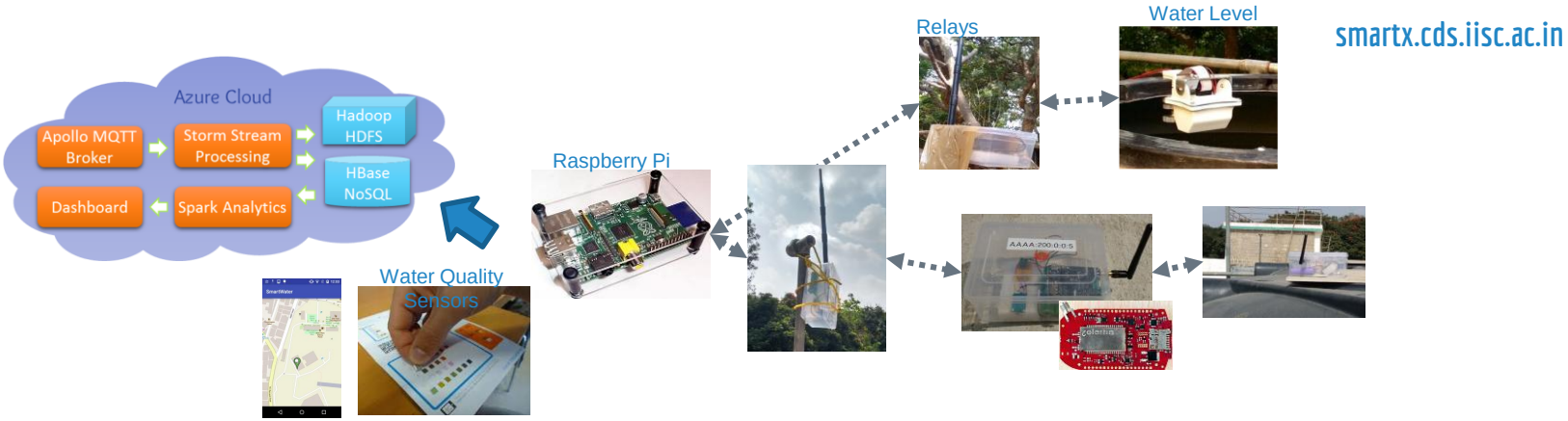
IBM

http://www.ibmbigdatahub.com/infographic/four-vs-big-data

Fast Data Analytics

IISc Smart Campus IoT Stack

- Hundreds of wireless motes & sensors
  - Realtime sensing of water level, quality, network health
  - Crowdsourced data collection from smart phones
  - *City-scale deployments will have even larger source and rates*
- Data analytics to drive online operational decisions
  - Water quality alert, water under/overflow, battery drain alert
  - *Smart power & transportation require even lower latency!*



Towards a Data-driven IoT Software Architecture for Smart City Utilities, Simmhan, Y., Ravindra, P., Chaturvedi, S., Hegde, M. and Ballamajalu, R., *Software: Practice and Experience,* 2018, Vol. 48(7), pp. 1390-1416,

IISc Campus

- 440 Acres, 8 Km Perimeter
- 50 buildings: *Office, Hotel, Residence, Stores*
- 10,000 people
- Water Use: 4M Lit/Day
- 10MW Power Consumed

Level Sensor

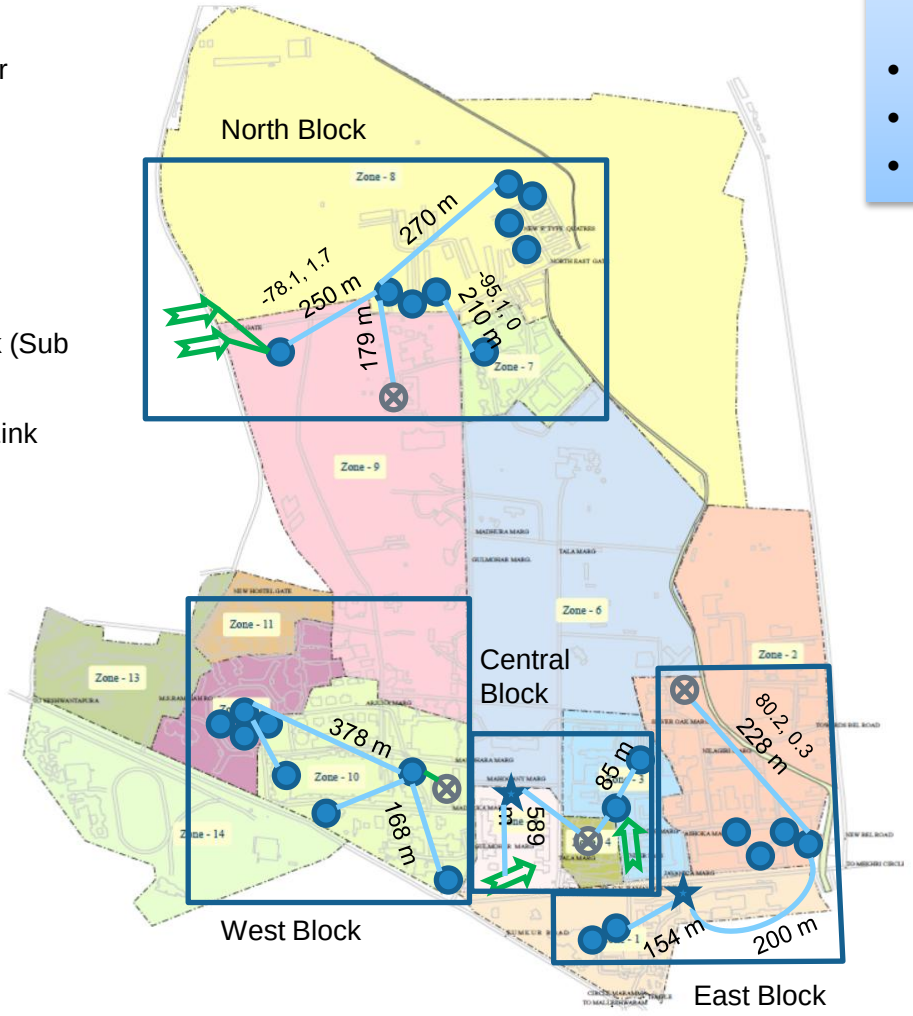
Flow Sensor

Relay

Gateway

Primary Link (Sub GHz)

Secondary Link (Zigbee)

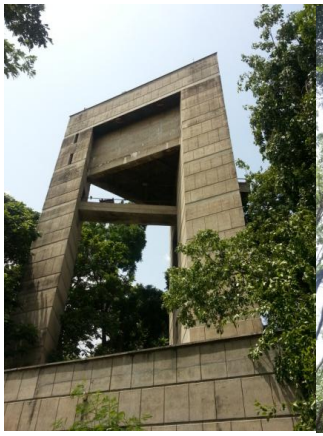


OHT	8
GLR	13
Inlet	4+3

Over Head Tanks (OHT)

TPH (near Mechanical)

E Type Quarters



Opposite to NESARA

Behind old C-Mess

Ground Level Reservoirs (GLR)

Opposite to CENSE

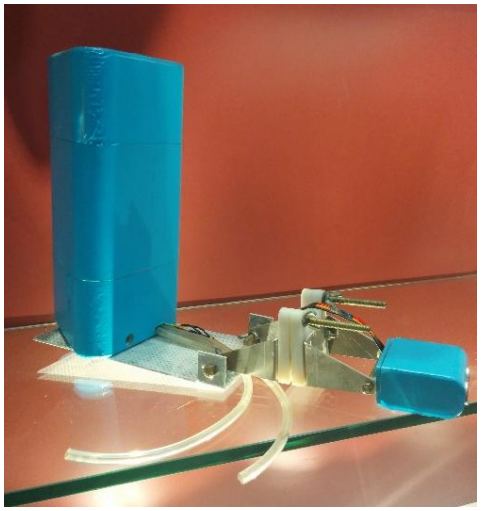
Boys Hostel

Near R Block

Near Janata Bazaar

Low-Cost Sensors

- Based on commodity H/W, in-house design, QC
  - Robust to external use
    - O(min) sampling
- Water level sensors
  - Current water quantity in OHT, GLR
  - Rate of inflow, outflow
- Water Quality
  - TDS, temperature
  - Physical, not chemical



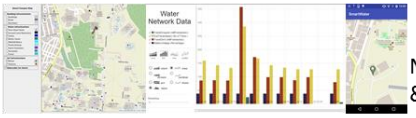
IoT for Smart Water Management

Water Infra.

Sensing

Scientist, Operator & Consumer Dashboards

Operation Decisions



Notification & Alerts

Smart Water Management Models

Demand Prediction | Leakage & Contamination Detection | Pumping Optimization

Scalable Big Data Framework on Edge+Cloud

Data & Event Analytics | Device Management & Policy

Scalable Data Archival | Information Ingest Pipeline & Stream Processing

Adaptive Network Infrastructure

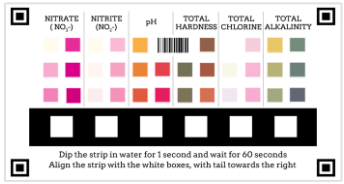
Design, Monitor, Manage Sensor N/W | SubGHz, LORA | Public N/W: WiFi, 3G/ 2G

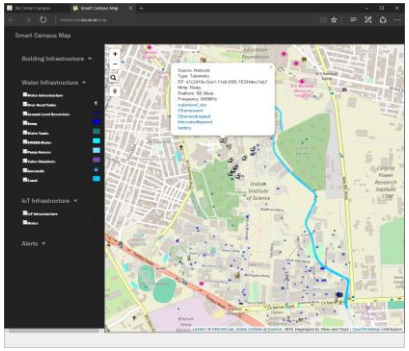
Crowd Sensing

Water Events | Quality Cards

Affordable Physical Sensing

Level Sensors | Quality Sensors | Flow Sensors

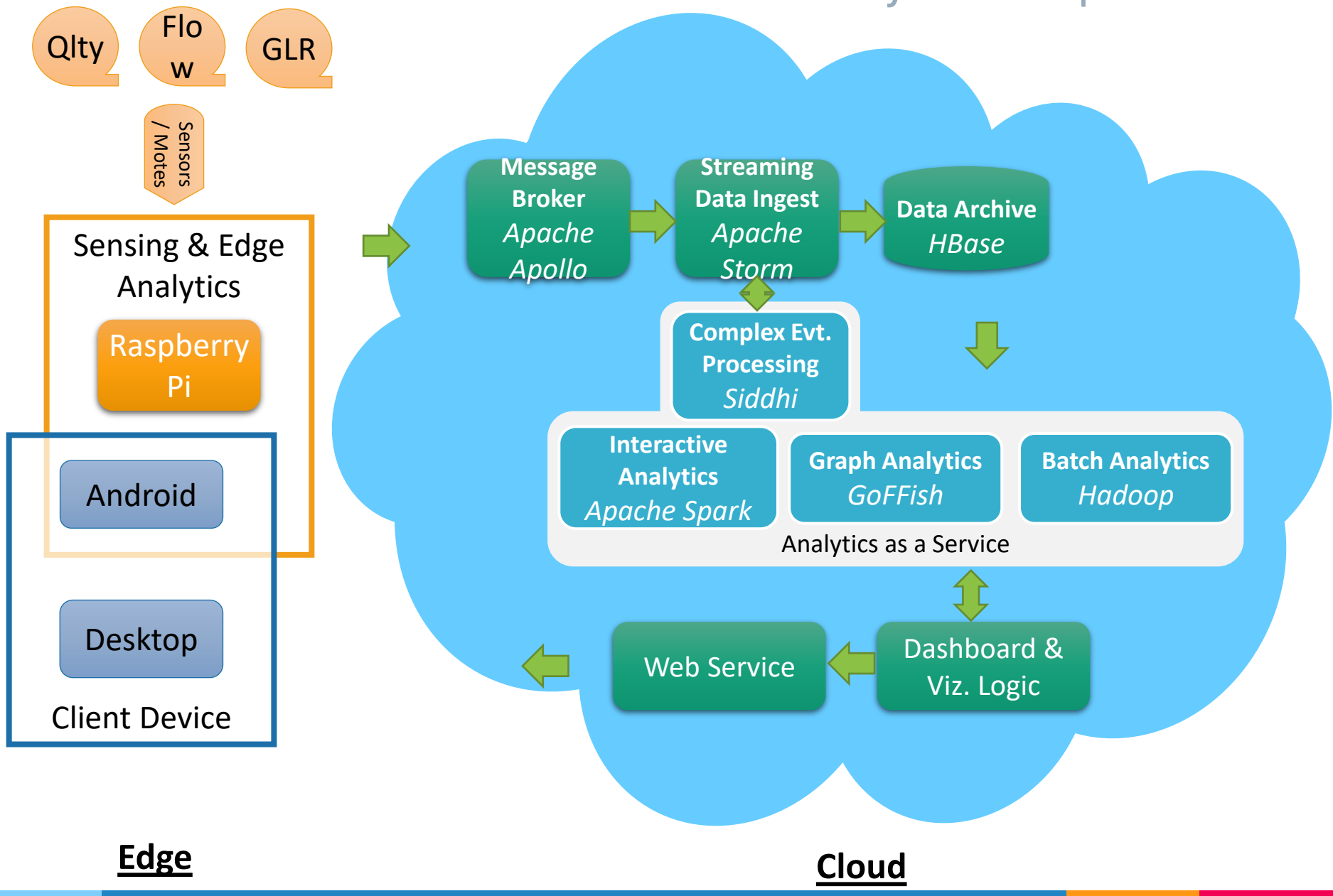




Analytics

Communic ation

Analytics Pipeline



IoT Software Stack

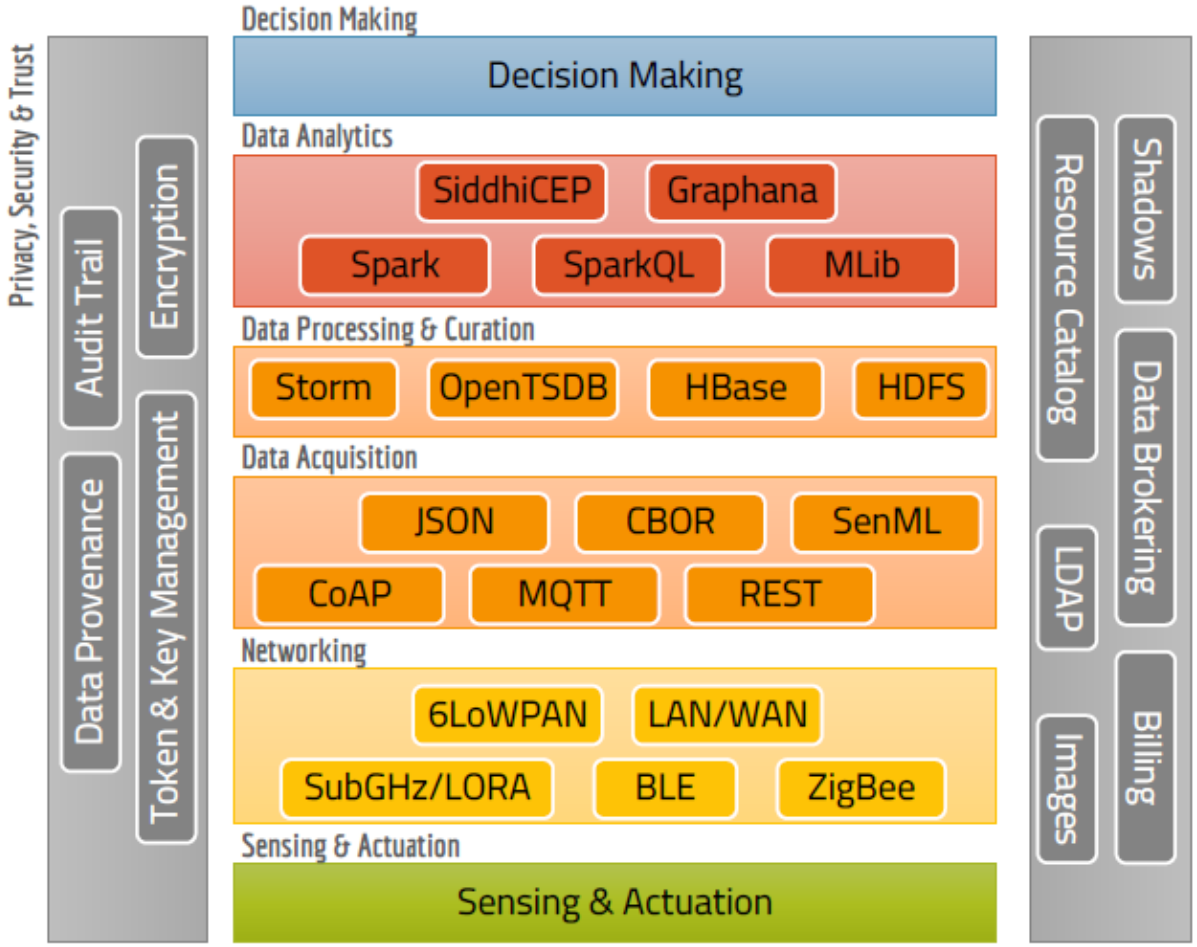
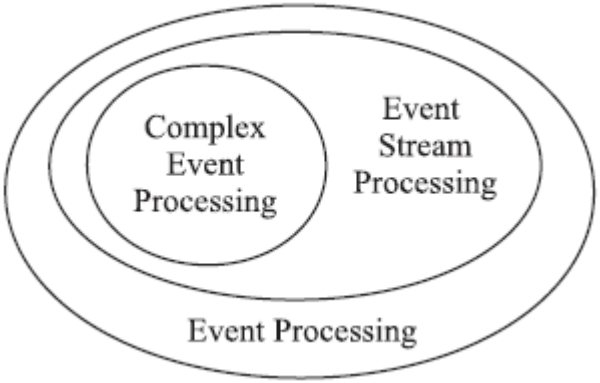
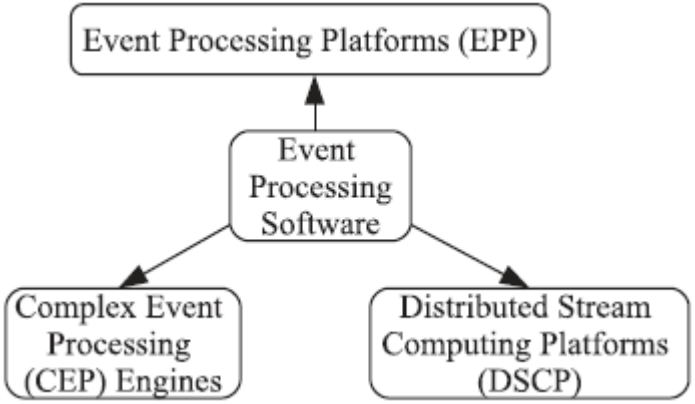
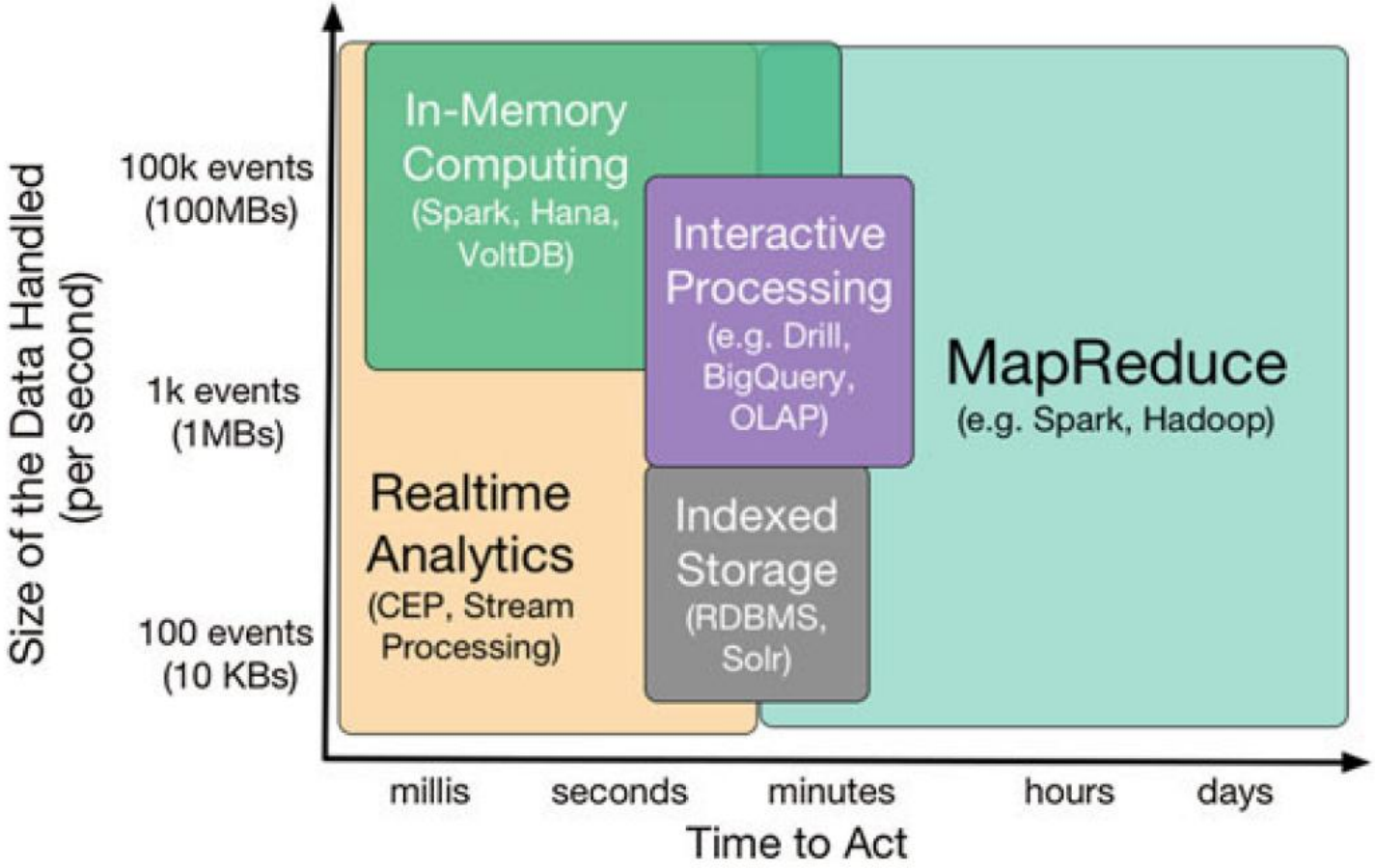


Figure 2: Protocol and Platform Architecture

Fast Data Ecosystem







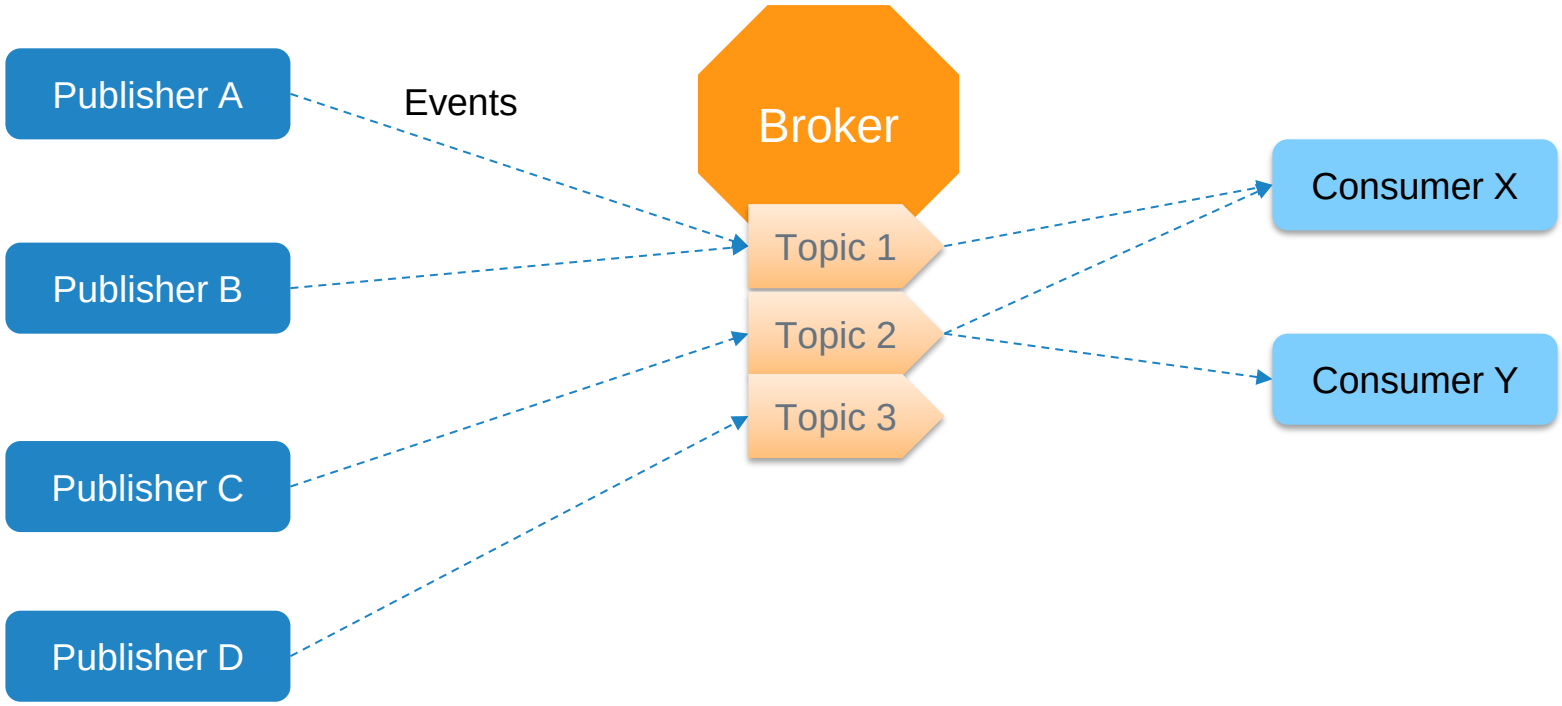
Kafka: Publish Subscribe Event Broker

Kafka, The Definitive Guide, Neha Narkhede, Gwen Shapira & Todd Palino, O'Reilly, 2017

Publish Subscribe System

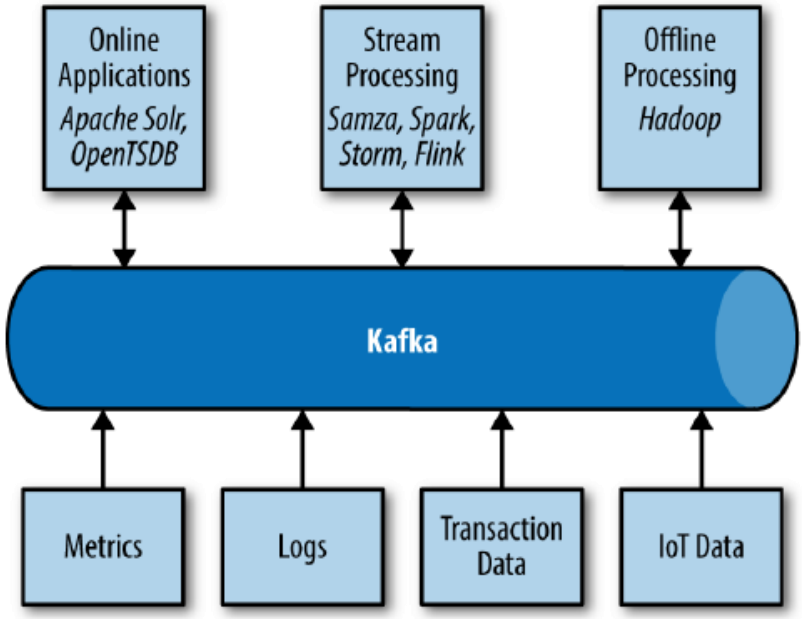
- Asynchronous Distributed Event-based Communication
- *Events* from some system
  - Data item of an observation or state, generated periodically or when some condition is met
- *Event Stream* if a series of events from same logical source
  - Generated by producers
  - Of interest to consumers
- What if many consumers are interested in the same event stream? Send data to each? How to scale?
- What if producers and consumers did not know of each other?
- What if consumers temporally went away?

Publish Subscribe System



Apache Kafka

- From LinkedIn
- For collecting system and application metrics
  - CPU usage and application performance
  - Request-tracing feature
  - Tracking user activity



- Decouple producers and consumers by using a push-pull model
- Provide persistence for message data within the messaging system to allow multiple consumers
- Optimize for high throughput of messages
- Allow for horizontal scaling of the system to grow as the data streams grew

Concepts

- Message
  - aka Event, Row, Tuple
  - Array of bytes
  - Key

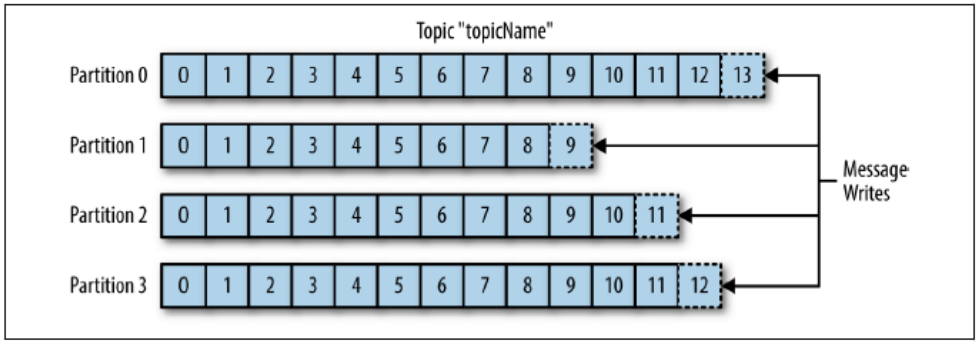


Figure 1-5. Representation of a topic with multiple partitions

- Topic
  - Like a table, or a PO Box
  - Sharded into partitions
  - Ordering preserved within a partition, not across
- Batch
  - Collection of messages for same partition
  - Grouping increases latency, improves thruput

Concepts

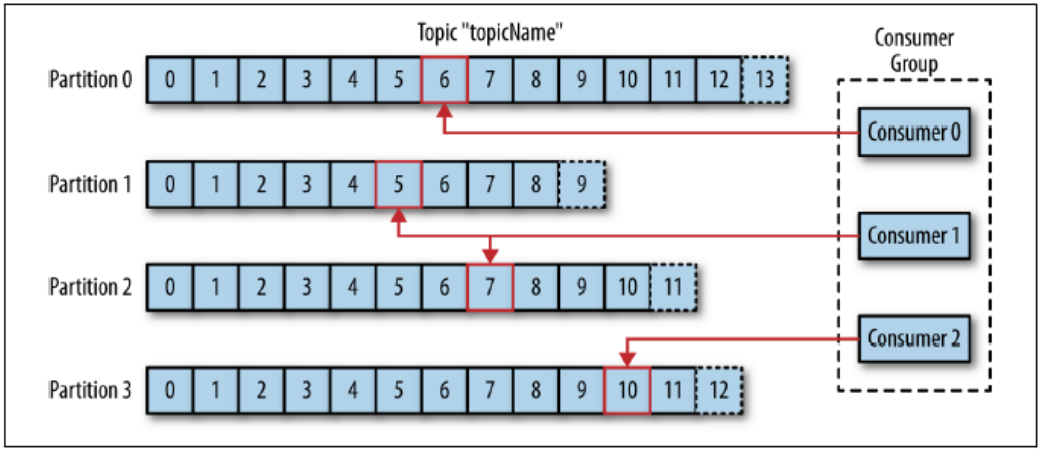


Figure 1-6. A consumer group reading from a topic

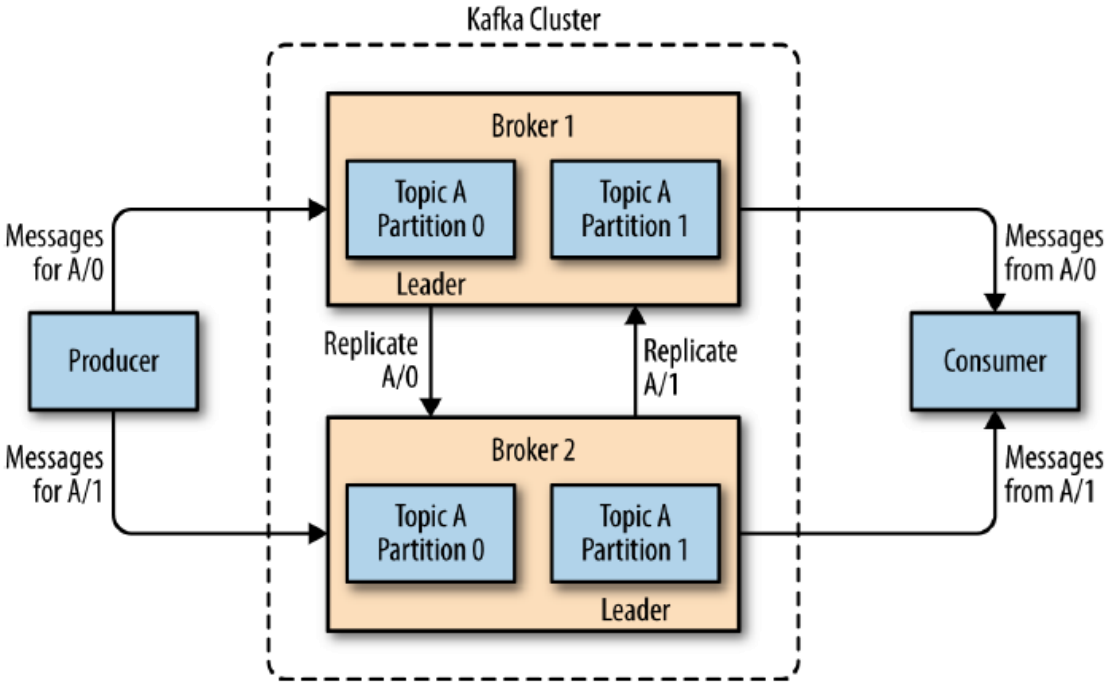
- Producers
  - create new messages
  - Sent to specific topic
  - May or may not* direct messages to a partition using key (and partitioner)
- Consumers
  - Read messages
  - Tracks messages consumed using an offset within a partition…allows robust restart of consumer on failure
- Consumer Group
  - Multiple consumers working together to consume messages from a topic
  - Each partition consumed only by one consumer
  - Horizontally scale, fault recovery, ordering within consumer for 1 partition

Concepts

- Broker
  - Single Kafka server
- From producer
  - receives messages from producers
  - assigns offsets to them within a partition
  - commits the messages to storage on disk
- To consumers
  - responds to fetch requests for partitions
  - Sends messages that have been committed to disk
- Message Retention
  - Durable storage of messages
    - Default time or time (e.g. 7 days, 1 GB)
  - Messages are expired and deleted after that

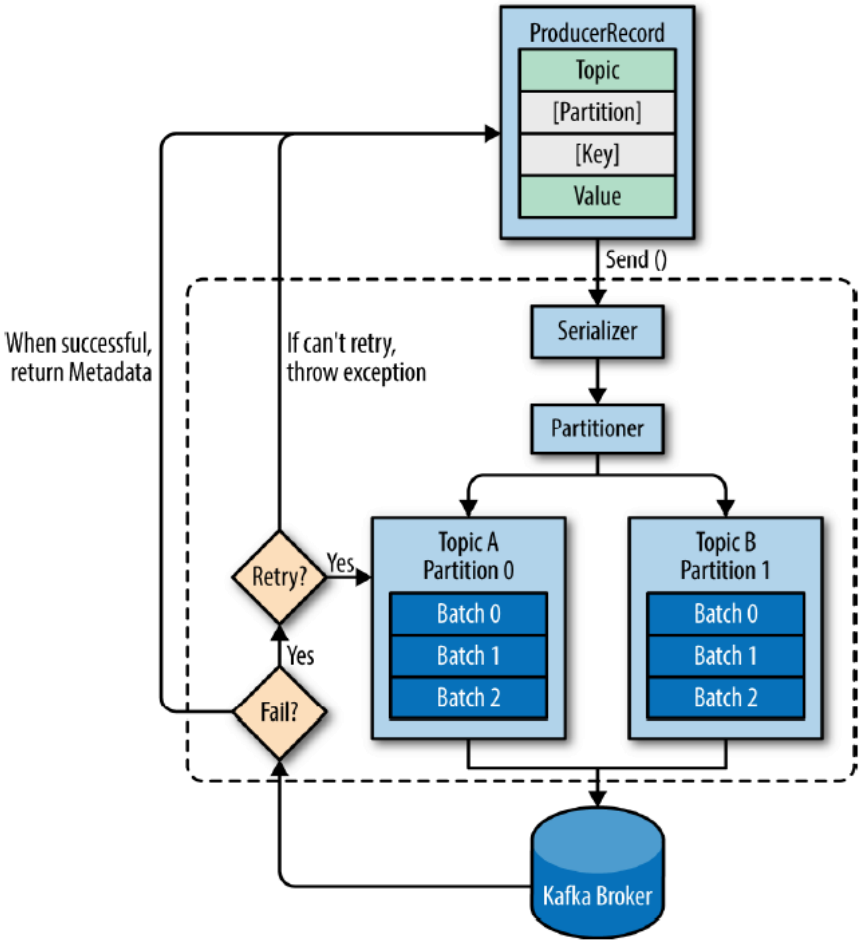
Kafka Cluster

- Horizontal scaling
- Partition owned by single broker (leader)
- Partition replicated to other brokers for redundancy if leader fails
- Producers and consumers only contact leader



Producer Workflow

- Message, Topic, Key*
- Serialize the message/key
- Send data to local partitioner, using key
- Batches messages for partition
- Sends batches to specific Kafka broker

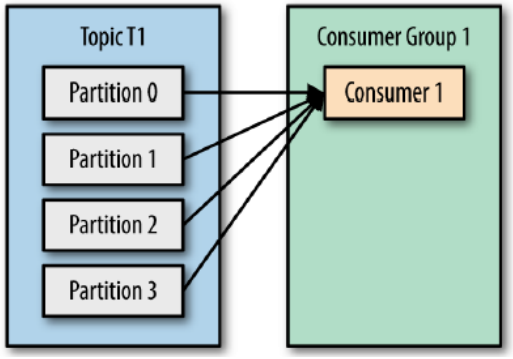


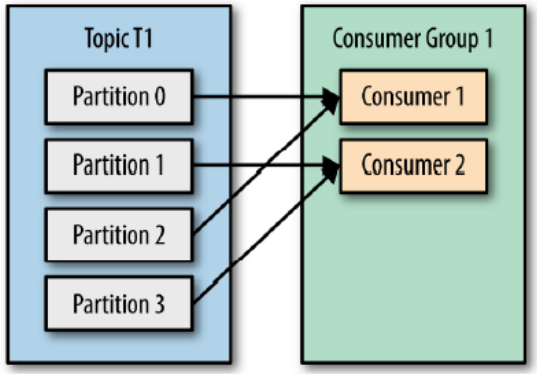
Sync vs Async

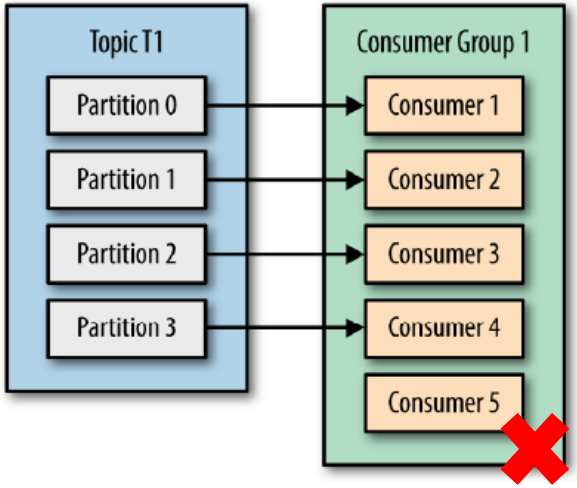
- Sync waits for ack from Kafka before sending next message
- topic, partition, and offset of the record written
- Async does not wait for replies
- Async with callback to handle errors

Consumer Workflow

- Subscribe to topics
- Partition mapped to one Consumer in Group
- More consumers than partitions keeps them idle
- Each consumer must be a separate thread
- Commit to tell broker that messages till a certain offset has been processed
  - Sync vs Async commit







Consumer Workflow

- Consumers join a group
  - First client in group is the leader
  - List of Consumers in a group are maintained by group coordinator
    - One of the brokers holding partitions for the topic

- Leader gets consumer list from coordinator, decides *assignment of partitions* to consumers
  - Range: Range of partitions in each topic subscribed by a consumer, e.g. P0-1 from Topic1, Topic2 for Cons 1, P2 from Topic1 Topic2 for Cons 2
  - Round Robin: Ensures each consumer in a group for one or more topics has almost the same number of partitions assigned, e.g. T1(P0,P2), T2(P1) for C1 and T1(P1), T2(P0,P2) for C2

Consumer Workflow

- Rebalance
  - Polling and commit ops serve as heartbeats to coordinator from a consumer
  - If a consumer fails to send heartbeat, its partition can be assigned to a different consumer in the group
    - i.e., Rebalance
  - If new partition is added, rebalance will be called
  - Need to commit pending operations on rebalance, using rebalance listener at consumer
- Standalone Consumer
  - Single consumer, not part of group
  - Can assign partition(s) to oneself to poll
  - Need to check for new partitions being added

Kafka Internals

- ZooKeeper Service
  - Maintaining configuration information
  - Naming, hierarchical namespace
  - Providing distributed synchronization (barrier, mutex, producer-consumer)
  - Providing group services
  - In-memory operations
  - Reads and watches are done locally on a znode by client, writes are through consensus, total global ordering of operations

Kafka Cluster Membership

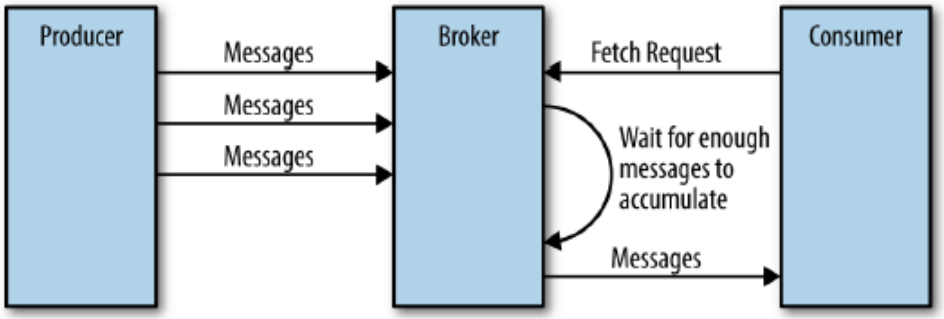
- Every broker creates ephemeral node
  - /brokers/ids
  - Ensures globally unique ID
  - Can subscribe to path, be informed if broker joins/goes away

- Controller
  - First broker, registers successfully at /controller
    - Others register watch, attempt to register themselves if earlier controller goes away
    - Controller epoch increments with new controller, skips stale operations of old controller epoch
  - Handles leader election for each replica by watching brokers
    - Informs leaders and follows upon selection

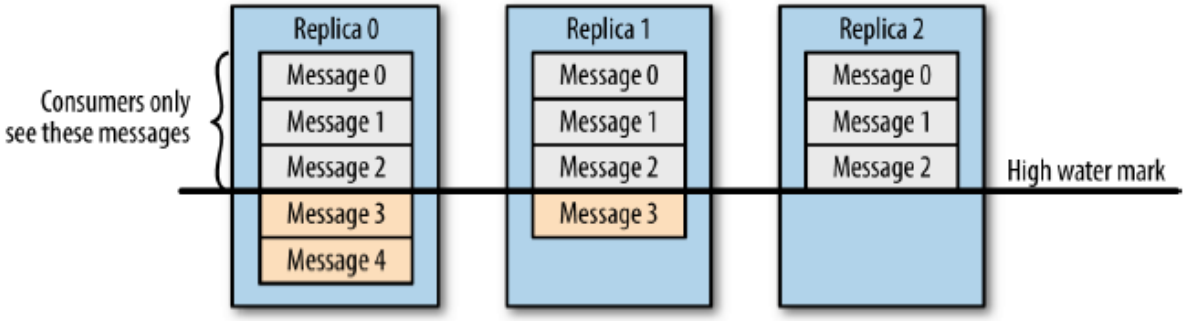
Kafka Leaders and Followers

- Leaders
  - Each partition has a single replica as the leader
  - All produce and consume requests go through the leader, in order to guarantee consistency
- Followers
  - replicate messages from the leader
  - stay up-to-date with the most recent messages
  - On leader failure, one of the follower replicas will be promoted as new leader
- Leader keeps track of sync status of each follower
  - Based on offset requests from follower
  - In sync are up to date, can be promoted as leader
  - Out of sync are <10secs old

Fetch Request



- Zero copy method for data from file to network directly, for a consumer
- Timeout on minimum number of messages accumulated, before being send to consumer
- Avoid sending “unsafe” messages, not yet been replicated to in-sync replicas



End of Lecture

- *Quiz on Module 4 on Thu Nov 9*

Complex Event Processing & Fast Data Querying

Complex Event Processing

- A form of Continuous SQL query over streams of events
  - Events can be seen as tuples in a relational table
  - Event Stream is like an unbounded table
- CEP uses patterns over sequences of simple events to detect and report composite events
- CEP can be thought of as an single “operator” in a stream processing system
- Or CEP can be thought of as a query language and platform for independent usage

Sample Event Streams and Queries

```
PowerStream<timestamp, location, consumptionKWH>
ACUnitStream<timestamp, location, unitId, temperature>
TankLevelStream<timestamp, building, tankId, levelCms>
StockTickStream<timestamp, scripCode, price>
```

Filters and Transformations

- This filters events from an input streams based on a predicate and places them, possibly after some transformation, into an output stream.

```
from PowerStream[consumptionKWH>10]
insert into HighPowerStream

from ACUnitStream#transform.Op(
(temperatureF-32)*5/9)
insert into ACUnitMetricStream
```

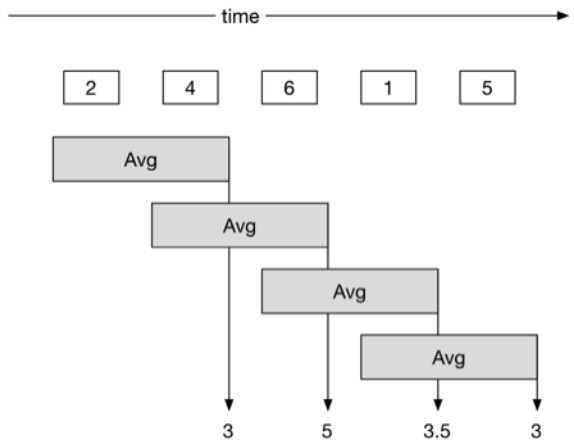
Window and Aggregation operations

- This operator collects data over a window (e.g., period of time or some number of events) and runs an aggregate function over the window.

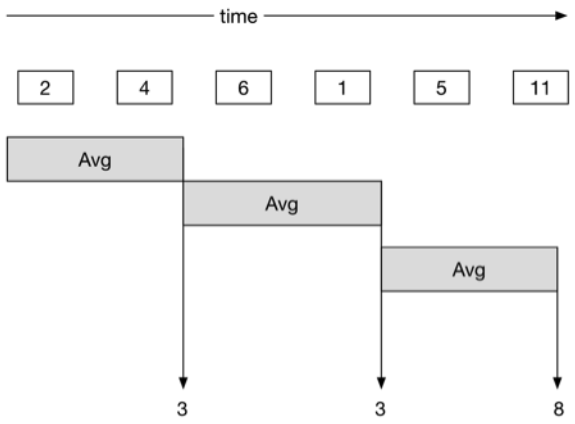
```
from PowerStream#window (60 min) select
avg (consumptionKWH)
```

Window Queries

- Sliding length window – keeps last N events and triggers for each new event.
- Batch length window – keeps last N events and triggers once for every N event.
- Sliding time window – keeps events triggered at last N time steps and triggers for each new event.
- Batch time window – keeps events triggered at last N time steps and triggers once for the time period at the end.



Sliding Windows

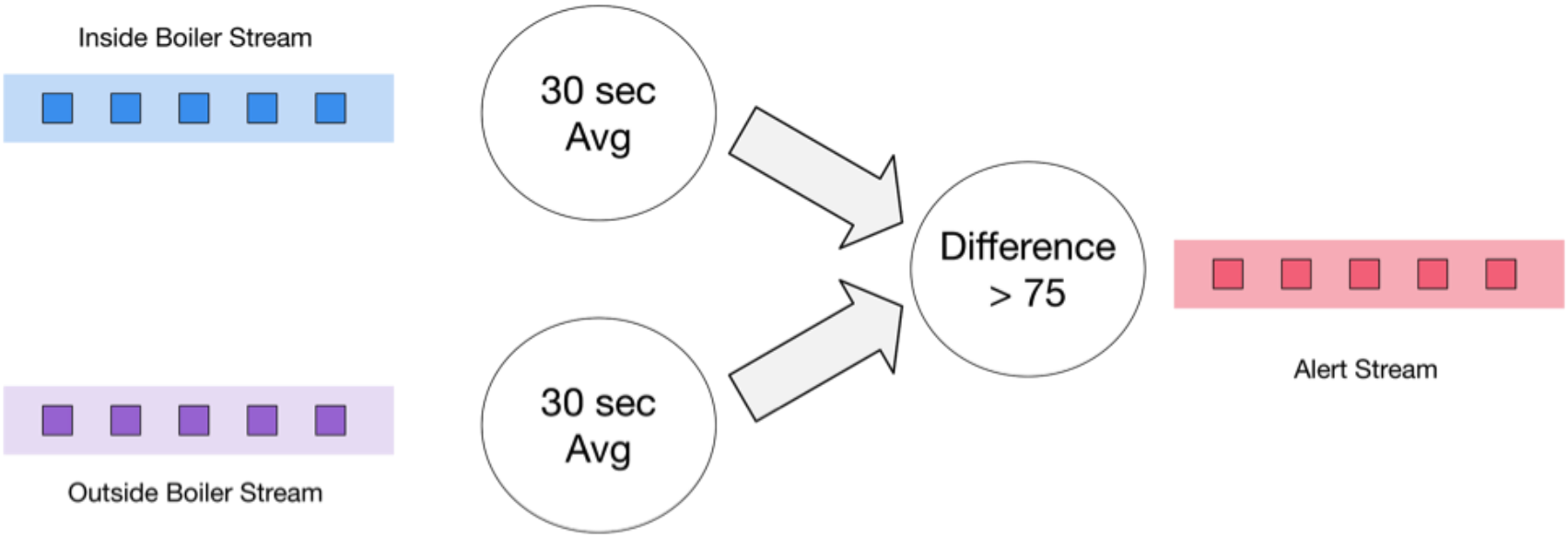


Batch Windows

https://wso2.com/library/articles/2018/02/stream-processing-101-from-sql-to-streaming-sql-in-ten-minutes/

Window Queries

- Form windows over two streams (30 sec.)
- Find average over each window
  - New avg. streams as result streams
- Join the avg streams and find the difference



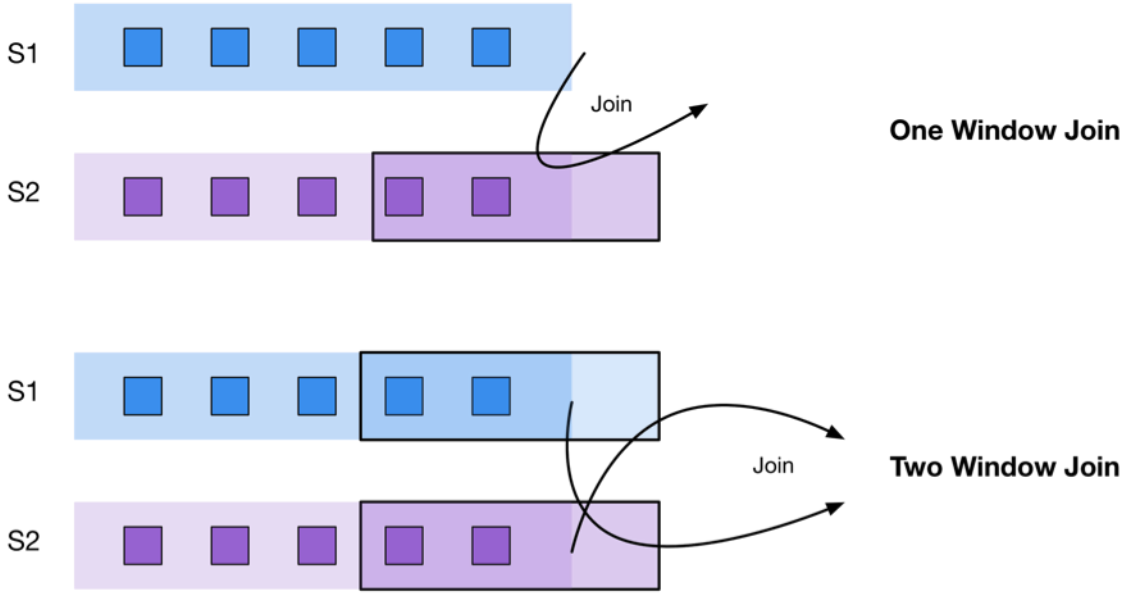
Join Queries

- This operator matches events from multiple data streams or data from the same stream based on a shared attribute value, e.g., location.
- This is similar to a database join, but performed on a window of events, e.g., of 1 min duration.

```
from PowerStream#window (1 min) join
ACUnitStream on PowerStream.location =
ACUnitStream.location
```

Join Window Execution

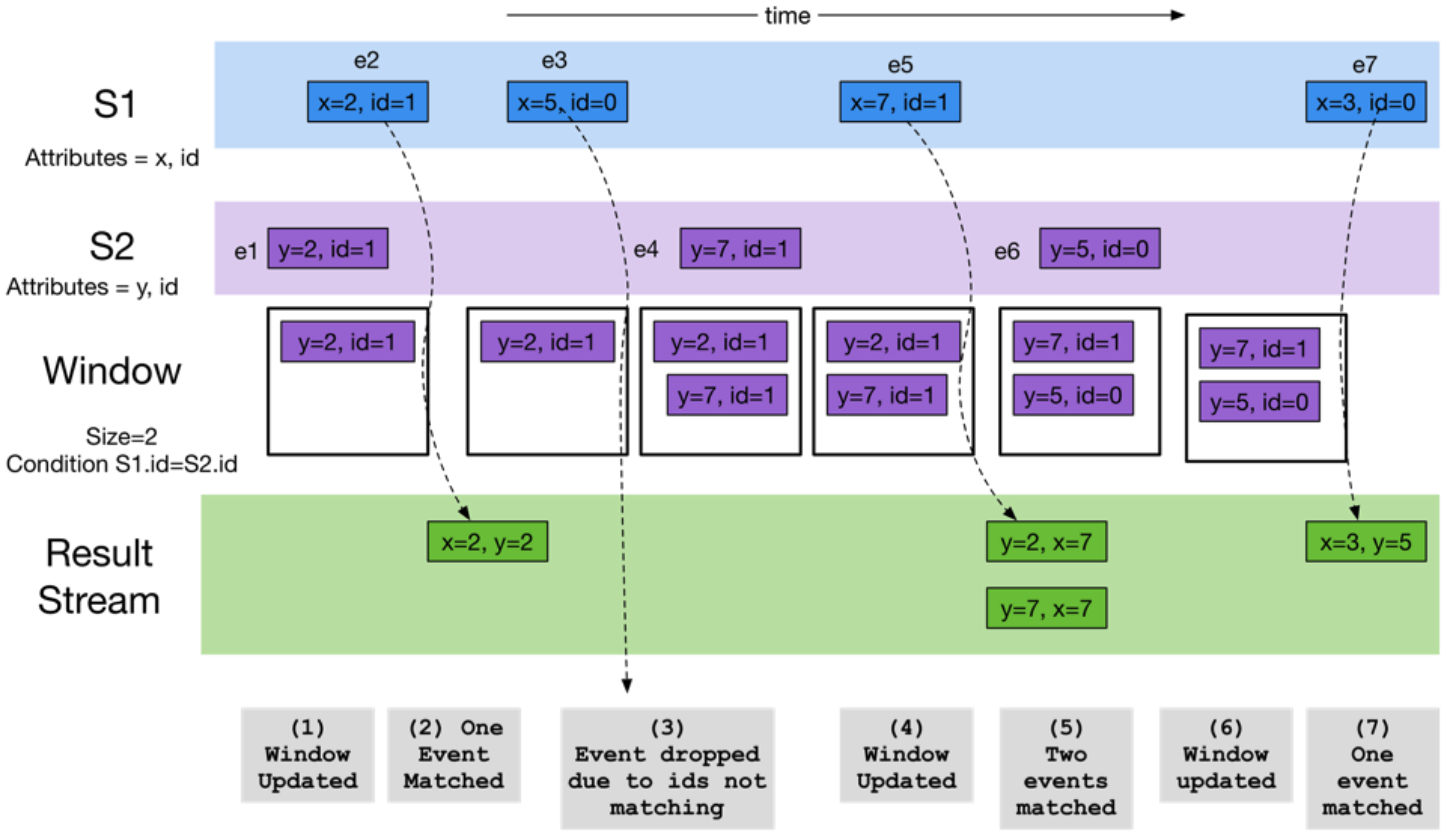
- One stream (S2) is bounded by a window, and events retained in the window are matched against events coming in the second stream (S1)
  - Only events in the second stream (S2) will trigger an output.
- A two-window query will retain events coming from both the streams in the window
  - A new event coming in will either trigger a match and an output.



https://wso2.com/library/articles/2018/02/stream-processing-101-from-sql-to-streaming-sql-in-ten-minutes/

Join Window Execution

```
Select x, y From S1 as s1 join S2.window(2) as
s2 on s1.id=s2.id insert into JoinedStream
```



https://wso2.com/library/articles/2018/02/stream-processing-101-from-sql-to-streaming-sql-in-ten-minutes/

Temporal sequence of event patterns

- Given a sequence of events, this detects a subsequence of events arranged in time where some property or condition is satisfied between the events
  - The consumption KWH between the events is greater than 10.
- This can also use regular expression matches between events, say, to detect a pattern of characters from a given sequence of characters.

```
from every e = PowerStream->
PowerStream[e.consumptionKWH -
consumptionKWH > 10]
insert into PowerConsumptionSpikeStream
```

More Examples from Smart Water Management

Query Type	Siddhi Query Definition

Filter	`define stream inStream (height int);`
	`from inStream[height<150] select height`
	`insert into outStream ;`

Sequence
Match 3 events
	`from every e1 = inStream, e2 = inStream[e1.height==`
	`e2.height], e3 = inStream[e3.height==e2.height]`
	`select e1.height as h1, e2.height as h2, e3.height as h3`
	`insert into outStream ;`

Pattern
Match 3 events
	`from every e1=inStream -> e2=inStream[e1.height==`
	`e2.height] -> e3=inStream[e2.height==e3.height]`
	`select e1.height as h1, e2.height as h2, e3.height as h3`
	`insert into outStream ;`

Aggregate (Batch) *Window Size = 60*	`from inStream #window.lengthBatch(60)` `select avg(height) as AvgHeight insert into outStream ;`
Aggregate (Sliding) *Window Size = 60*	`from inStream #window.length(60)` `select avg(height) as AvgHeight insert into outStream ;`

CEP Engines

- Takes in input stream sources, queries, performs matching
- Can be standalone or embedded
- Different query models, including SQL-like
- Sample Platforms
  - Siddhi, Esper, etc.

Comparison of Event Processing Technologies

Table 2. Feature Comparison of Event Processing Platforms

Feature	TIBCO Stream-Base	IBM Infosphere Streams	DataTorrent RTS	Amazon Kinesis	WSO2 CEP	Fujitsu Interstage CEP	SQLstream
SQL-like queries	●				●		●
Query composition	●	●	●	●	●		●
Temporal Pattern and sequence queries	●		●		●	●	
Key query types: Joins (J), Windows (W)	J/W	J/W	J/W		J/W		J/W
Window types: Sliding (S), Time (I), Tuple (T), Batch (B)	S	S / I / T / B	S / I / B		S / I / T[3] / B		S / I / B
Integrate with GIS Servers?	–	–				●	
Show results	●	●	●	●			●
DB Integration	●	●	●		●	●	●
Distributed Processing	●	●	●	●	●		●
Visual Application Debugger?	●	●	●		●		●
Messaging system support: JMS (M)/ Kafka (K)	M/K	M	M/K		M/K	M	M/K
Built-in Fault Tolerance[2]		●	●	●[4]			
High availability	●	●	●	●	●		
Scalable?	●	●		?[5]	●		?[5]
Best reported performance	91,000 evals/s [186]	7.3 Million Tuples/s [132]	76 Million tuples/s (moving average) [158]	–	0.4 Million events/s [151]	upto 2 Million events/s [70]	4.68 Million records/s [178]
Comprehensive operator library	●	●	●		●		●
API support/ language extensions for geoinformation processing	–	–		●	●	●	
Open source license: Apache (A), GNU GPL (G), BSD License (B)	–	–	A[1]	–	A	–	–

Comparison of Stream Processing Technologies

Table 3. Feature Comparison of Distributed Stream Computing Platforms

Feature	S4	Storm	Spark Streaming	Samza	Apex	Flink
SQL-like queries		●	●			●
DB Connection	●	●	●	●	●	●
Relational DB	●	●	●	●	●	●
Key query types: Joins (J), Windows (W)	J, W	J	J, W	W	J, W	J, W
Window types: Sliding (S), Time (I), Tuple (T), Batch (B)	S	—	S, I, T, B	I	S, I, T, B	S, I, T, B
Fault tolerance	—	Exactly-once	Recompute	State persistence	State persistence	Exactly once/at least once
HA	●	●	●[1]	●	●	●
Scalable?[2]	●	●	●		●	●
Best reported performance	Up to 10 thousand events/s [58]	3.2 Million tuples/s [146]	130,000 events/s [162]	1.2 Million messages/s [68]	3.3 Million tuples/s [159]	15 Million events/s [85]

Comparison of CEP Engines

Table 4. Feature Comparison of Complex Event Processing Libraries

Feature	Esper (Basic Version)	Siddhi	RuleCore	Cayuga
SQL-like queries	●	●	●	
DB connection	●	●		
Open source license: Apache (A), GNU GPL (G), BSD License (B)	G	A	G	B
Debugging support	●			
Window types: Sliding (S), Time (I), Tuple (T), Batch (B)	I, T, B	S, I, T, B	T, S	T, S
sequences	●	●	●	
Event Aggregation	●	●		●
Nested queries	●			
Data extraction	●	●		
Parameterization	●	●		
Best-reported performance	500 000 event/s [63]	8.55 Million events/s [150]	—	—

Additional Reading

- A Gentle Introduction to Stream Processing, Srinath Perera
  - https://medium.com/stream-processing/what-is-stream-processing-1eadfca11b97
- Big Data Analytics Platforms for Real-time Applications in IoT, Simmhan Y., Perera S. (2016) In: Pyne S., Rao B., Rao S. (eds) Big Data Analytics. Springer, New Delhi. https://doi.org/10.1007/978-81-322-3628-3_7
- Dayarathna, Miyuru, and Srinath Perera. "Recent advancements in event processing." *ACM Computing Surveys (CSUR)* 51.2 (2018): 1-36.
- Nasiri, Hamid, Saeed Nasehi, and Maziar Goudarzi. "Evaluation of distributed stream processing frameworks for IoT applications in Smart Cities." *Journal of Big Data* 6.1 (2019): 1-24.
- Shukla, Anshu, Shilpa Chaturvedi, and Yogesh Simmhan. "Riotbench: An iot benchmark for distributed stream processing systems." *Concurrency and Computation: Practice and Experience* 29.21 (2017): e4257.
- Cugola, Gianpaolo, and Alessandro Margara. "Processing flows of information: From data stream to complex event processing." *ACM Computing Surveys (CSUR)* 44.3 (2012): 1-62.

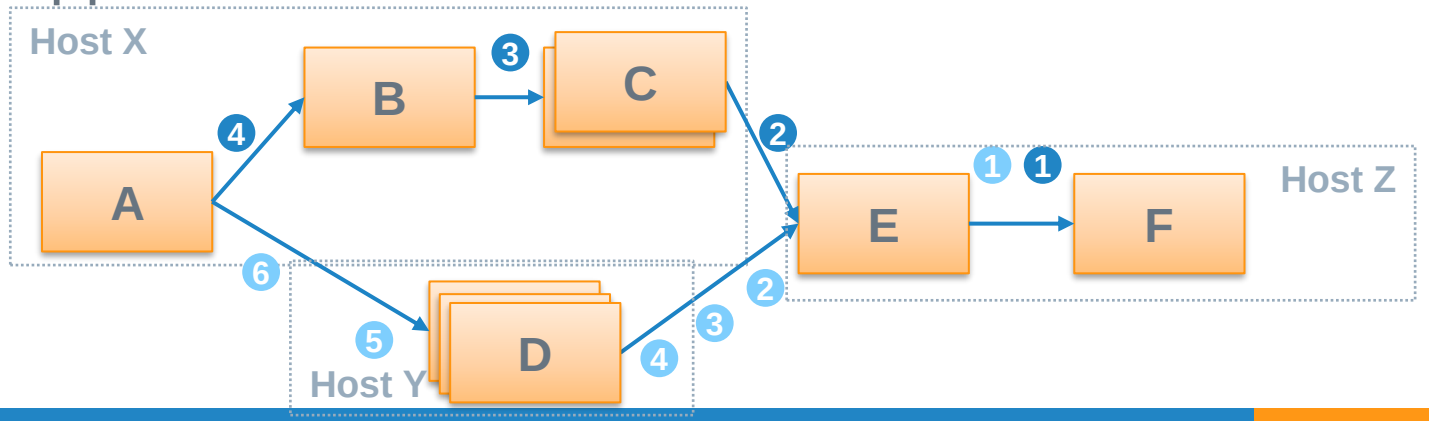
Spark Streaming for Fast Data Processing

Learning Spark, 2nd Edition, Chapter 8

Discretized Streams: Fault-Tolerant Streaming Computation at Scale, Zaharia et al., *SOSP* 2013

Distributed Stream Processing

- User application composed as a dataflow
  - Logic blocks (tasks) consume one event and generate zero or more events
  - Limited support for *stateful execution*, *routing semantics*
- Tasks operates on events independently, exploit parallelism
  - **Data-parallel** execution across events, streams [C, D]
  - **Task-parallel** execution over parallel dataflows [B,C||D]
  - **Pipelined** execution [A, D, E, F]
- Typically, *no global ordering guarantee*. Stream *joins* are not well-supported. *Interleaved*.



Distributed Stream Processing

- E.g., Storm, Flink, etc.

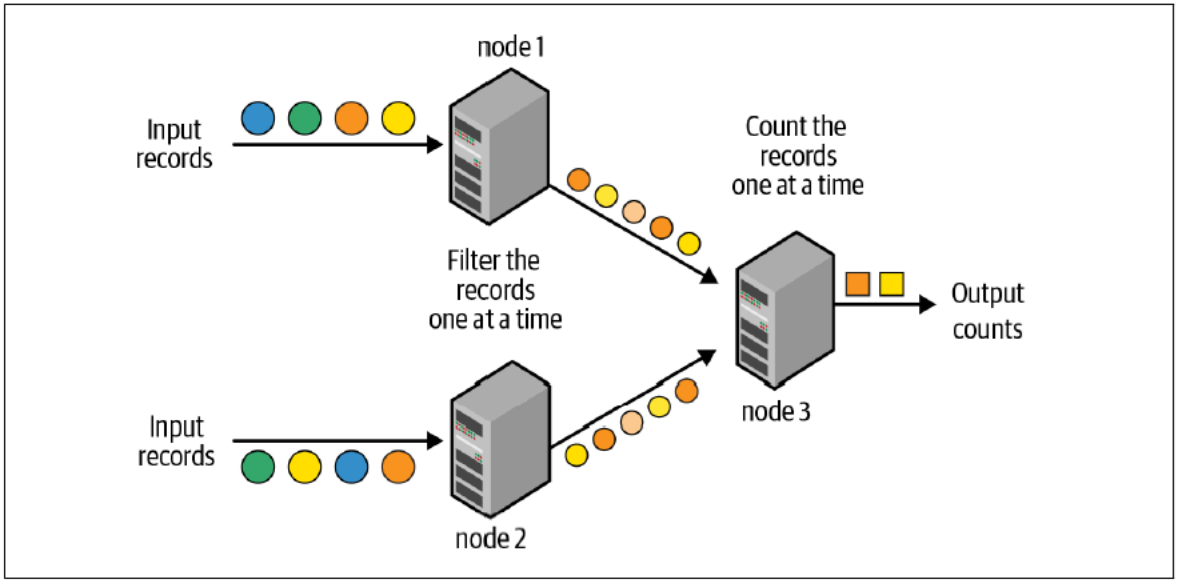


Figure 8-1. Traditional record-at-a-time processing model

Distributed Micro-Batch Stream Processing

- Since Spark 1.x

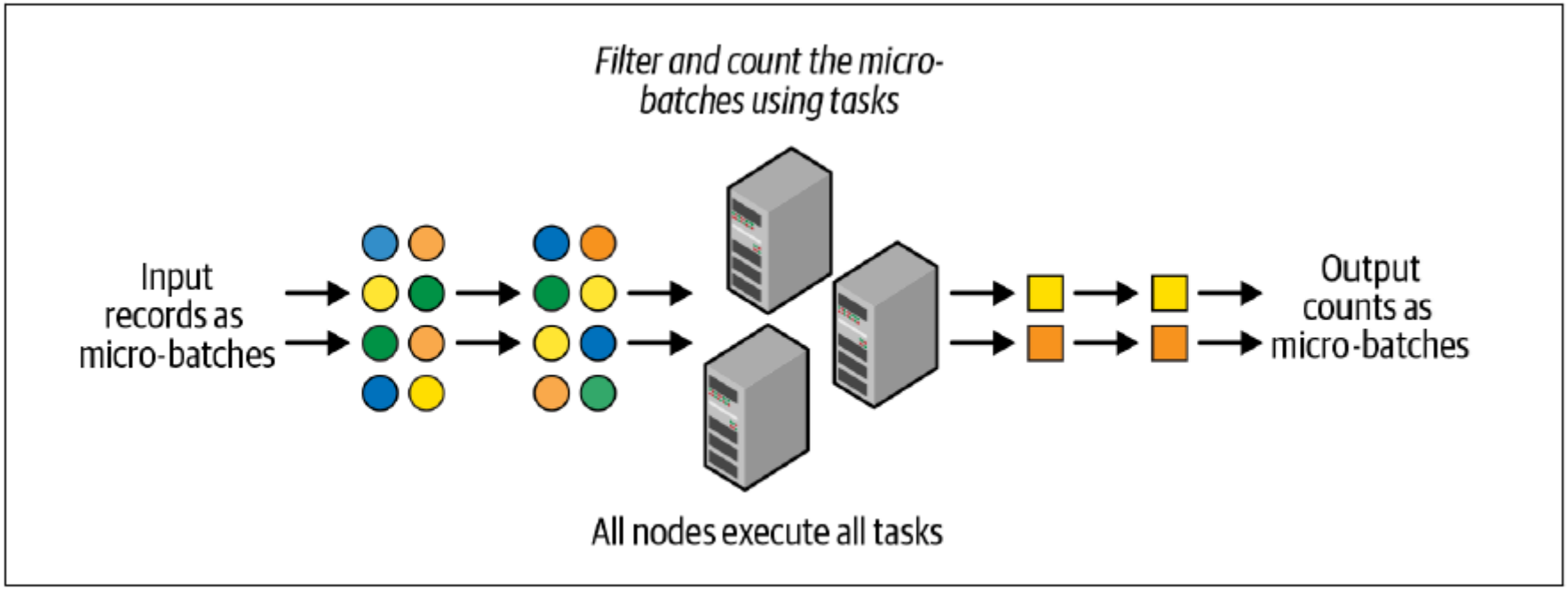
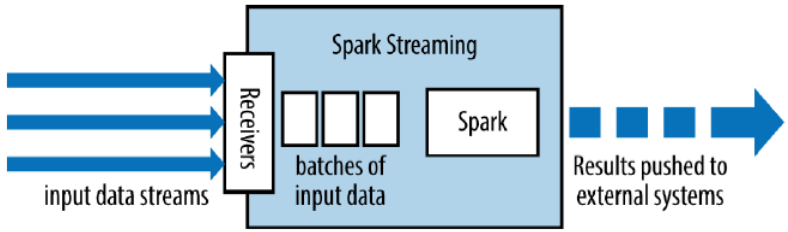


Figure 8-2. Structured Streaming uses a micro-batch processing model

Discretized Streams (DStream)

- **DStream** is a sequence of data arriving over time
  - Represented as a sequence of **RDDs** arriving at each time step
  - Can be created from Kafka, HDFS, etc.
- *Transformation operations*
  - Yields a new DStream
- *Output operations*
  - Write to an external system
- But similar limitations as RDD
  - And no support for event windows (rather than processing windows)



Structured Streaming

- Model a stream as an “unbounded” table/dataframe
  - Unified API for batch and stream processing
  - Continuum between real-time and batch
- Spark 2.x

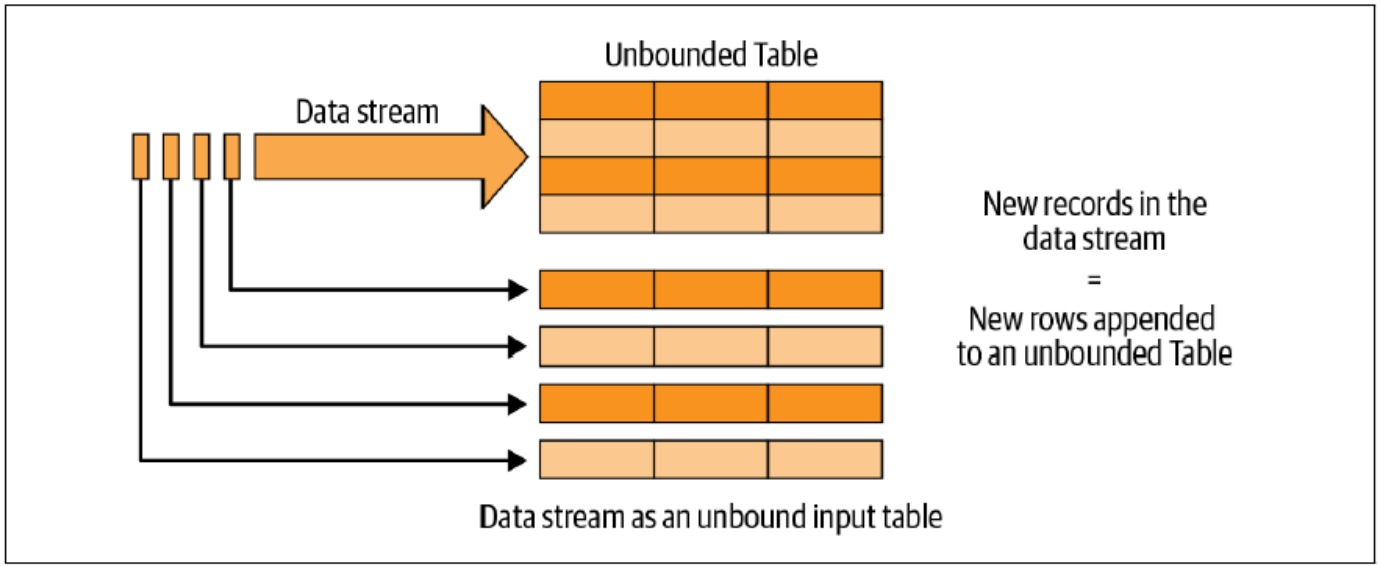


Figure 8-3. The Structured Streaming programming model: data stream as an unbounded table

Structured Streaming

- Incremental execution of “batch” queries over dynamic DataFrame

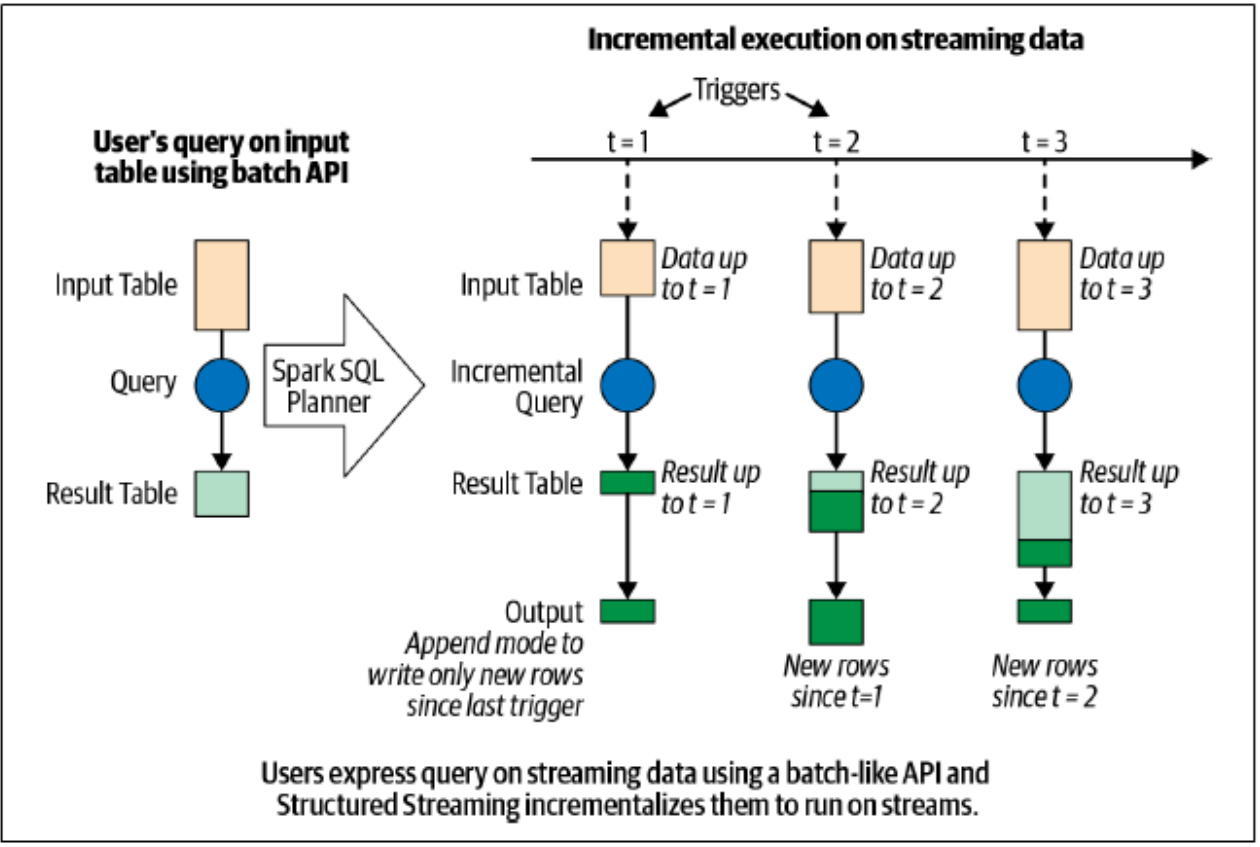
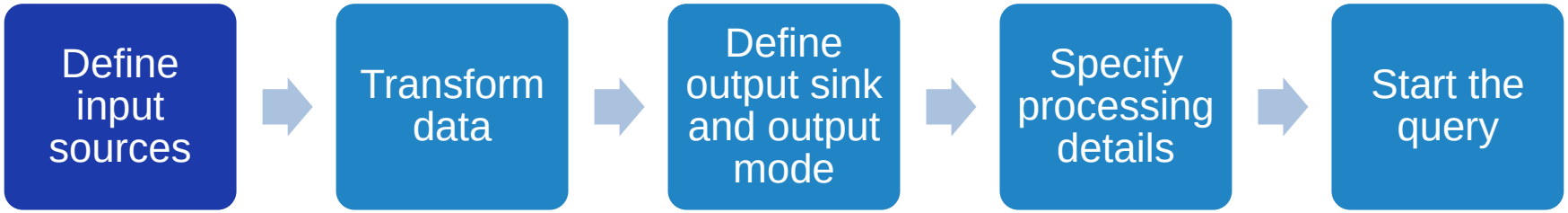


Figure 8-4. The Structured Streaming processing model

- Trigger based on time
- Output table is Appended to, Updated or Replaced

Streaming Query Lifecycle

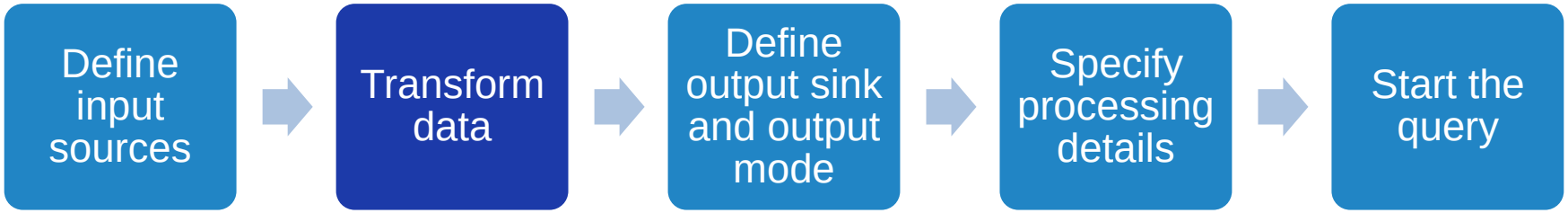


- Define input stream
  - Used to populate and update input dataframe
  - Can be an unbounded stream

```
# In Python
spark = SparkSession...
lines = (spark
  .readStream.format("socket")
  .option("host", "localhost")
  .option("port", 9999)
  .load())
```

```
# In Python
inputDF = (spark
  .readStream
  .format("kafka")
  .option("kafka.bootstrap.servers", "host1:port1,host2:port2")
  .option("subscribe", "events")
  .load())
```

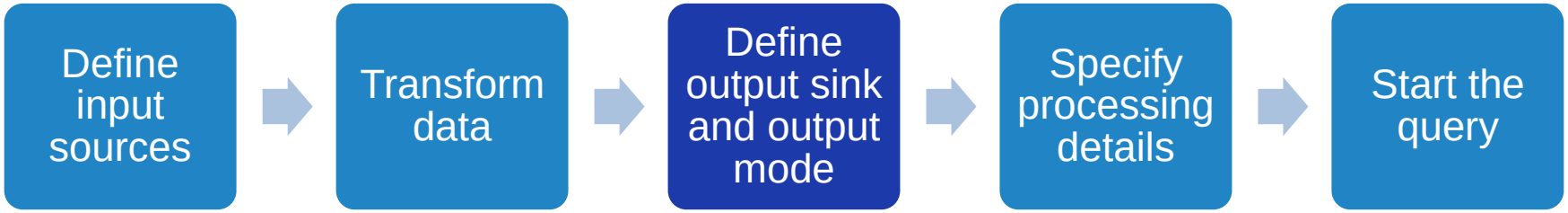
Streaming Query Lifecycle



- Transform data using standard DF API
  - **Stateless operations** only require row at a time: select(), filter(), map()
  - **Stateful operations** require prior state/multiple rows: count()

```
from pyspark.sql.functions import *
words = lines.select(split(col("value"), "\\s").alias("word"))
counts = words.groupBy("word").count()
```

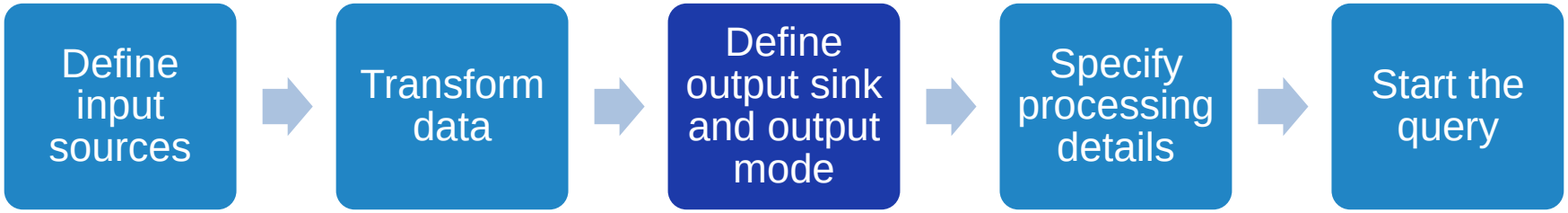
Streaming Query Lifecycle



- Define output sink (Console, File, Kafka), and what to emit
  - **Append**: Only new rows added to the output DF since the last trigger will be output to the sink. Only stateless queries that never modify previous rows.
  - **Update**: Only rows that are modified in output DF are send to sink
  - **Complete**: All rows of output DF are output to sink. Results table should remain small over time.

```
# In Python
writer = counts.writeStream.format("console").outputMode("complete")
```

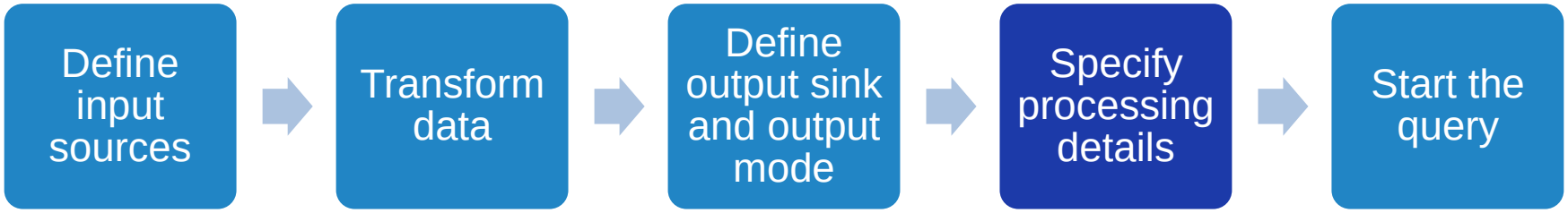
Streaming Query Lifecycle



- Writing to Kafka topic

```
counts = ... # DataFrame[word: string, count: long]
streamingQuery = (counts
  .selectExpr(
    "cast(word as string) as key",
    "cast(count as string) as value")
  .writeStream
  .format("kafka")
  .option("kafka.bootstrap.servers", "host1:port1,host2:port2")
  .option("topic", "wordCounts")
  .outputMode("update")
  .option("checkpointLocation", checkpointDir)
  .start())
```

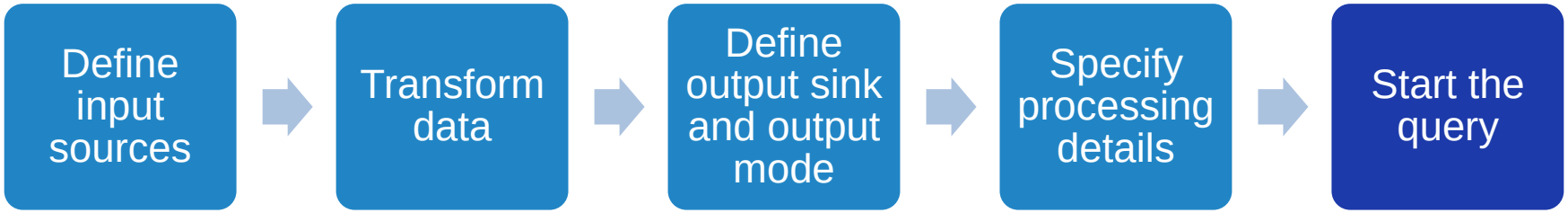
Streaming Query Lifecycle



- Triggering of execution on input DF
  - **Default**: Executes data in micro-batches. Next micro-batch starts after previous micro-batch finishes.
  - **Trigger Interval**: Accumulates new rows for 'n' seconds into a micro-batch and executes each.
  - **Once**: Executes one micro-batch with all available data.
  - **Continuous**: Executes one row at a time (v3)
- **Checkpointing** to persistent store allows exactly once guarantees on input stream

```
# In Python
checkpointDir = "..."
writer2 = (writer
  .trigger(processingTime="1 second")
  .option("checkpointLocation", checkpointDir))
```

Streaming Query Lifecycle



- Initiate execution of streaming execution
  - start() on writeStream is non blocking
  - Can stop() the streamingQuery or awaitTermination()

```
# In Python
streamingQuery = writer2.start()
```

Full Example

```
# In Python
from pyspark.sql.functions import *
spark = SparkSession...
lines = (spark
  .readStream.format("socket")
  .option("host", "localhost")
  .option("port", 9999)
  .load())

words = lines.select(split(col("value"), "\\s").alias("word"))
counts = words.groupBy("word").count()
checkpointDir = "..."
streamingQuery = (counts
  .writeStream
  .format("console")
  .outputMode("complete")
  .trigger(processingTime="1 second")
  .option("checkpointLocation", checkpointDir)
  .start())
streamingQuery.awaitTermination()
```

Execution Model

- Create micro-batch, optimize logical plan for execution, update result, emit to output sink

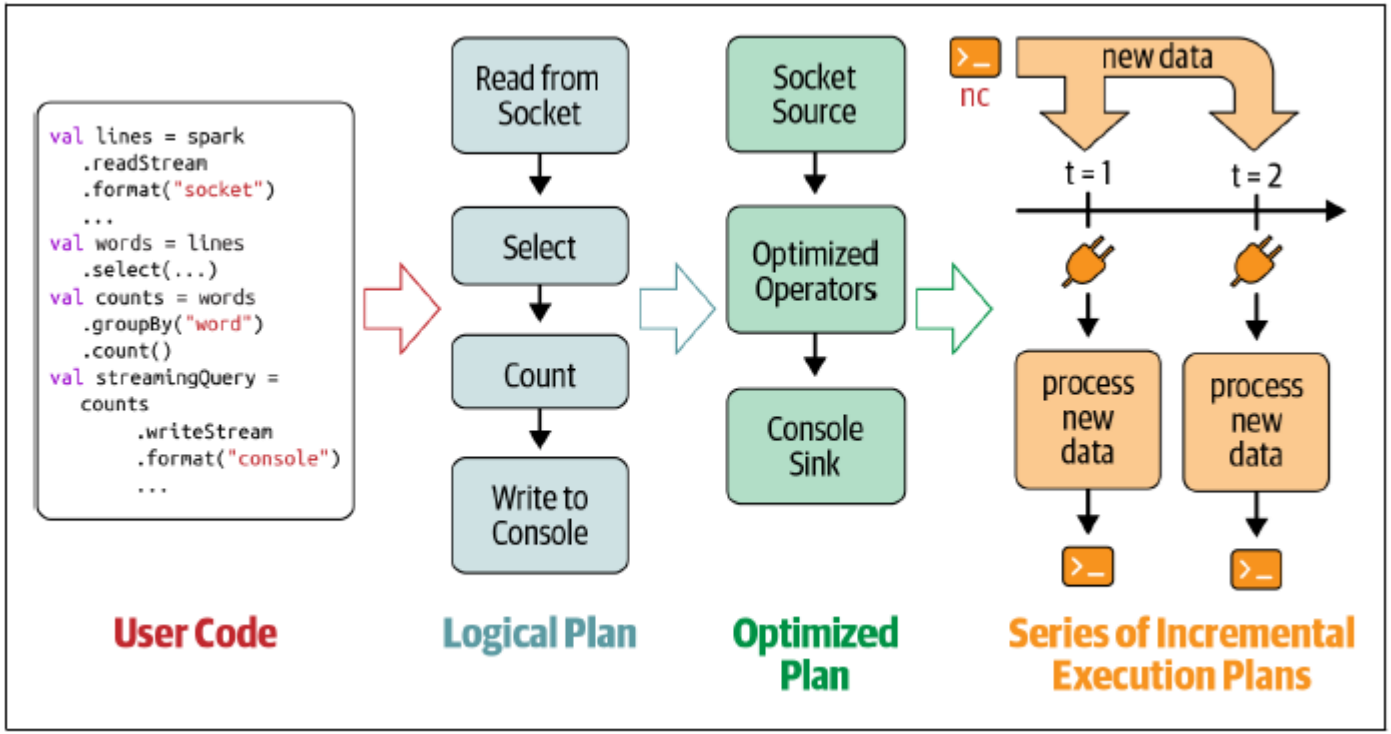


Figure 8-5. Incremental execution of streaming queries

Stateless and Stateful Transformations

- A streaming query having only *stateless* operations supports the **append** and **update** output modes, but not complete mode
- *Stateful* transformations perform incremental aggregation
  - Need to ensure state is maintained across micro-batches
  - Distributed processing gives correct result
  - Has to use **groupBy** clause

Stateful Distributed Execution

- *Partitioning* and *shuffles* are done consistently to ensure same **executor** operates on same "groupBy" column values
  - External storage to maintain states for fault recovery

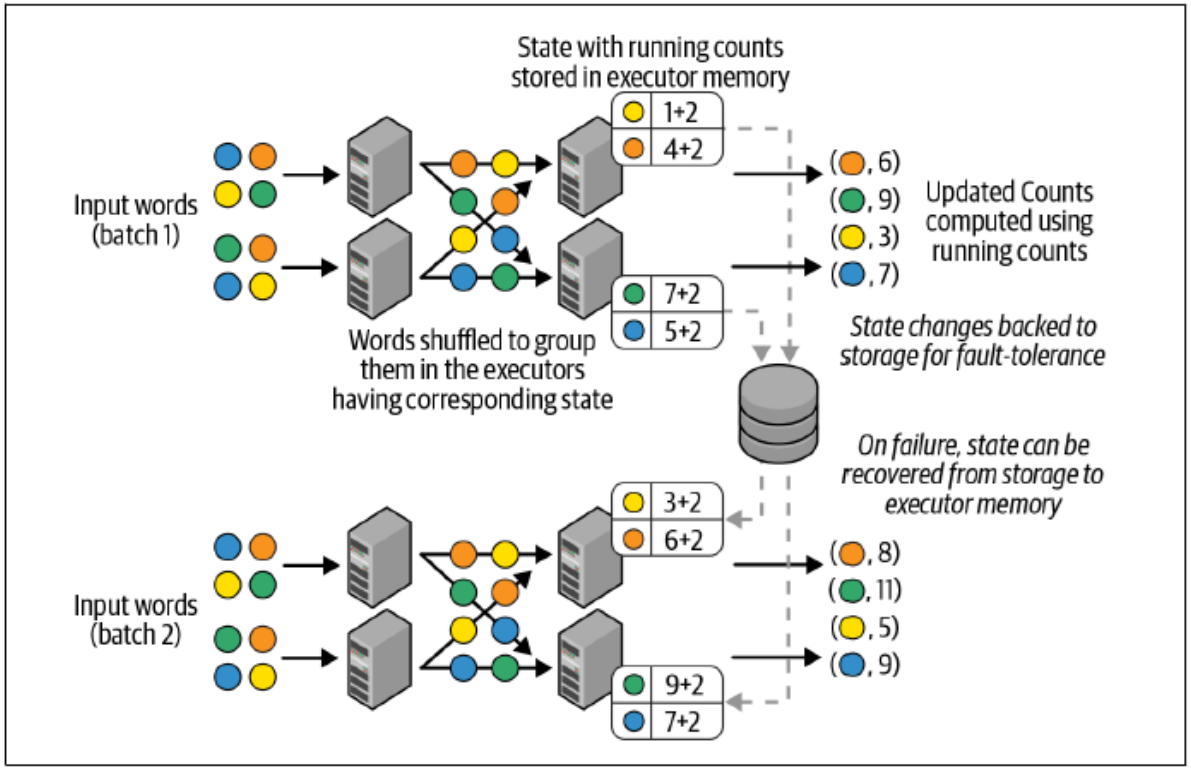


Figure 8-6. Distributed state management in Structured Streaming

Non-temporal Aggregations

- Aggregation since the start of stream
- Global Aggregation

```
# In Python
runningCount = sensorReadings.groupBy().count()
```

- Grouped Aggregation, running aggregate by column value

```
# In Python
baselineValues = sensorReadings.groupBy("sensorId").mean("value")
```

- Common built-in aggregation functions
  - sum(), mean(), stddev(), countDistinct(), collect_set(), approx_count_distinct()
- Chaining Aggregators

```
# In Python
from pyspark.sql.functions import *
multipleAggs = (sensorReadings
  .groupBy("sensorId")
  .agg(count("*"), mean("value").alias("baselineValue"),
    collect_set("errorCode").alias("allErrorCodes")))
```

Aggregations with Event-Time Windows

- Aggregations over data bucketed by time windows
- Use the *event time* (timestamp in row) rather than *processing time* (when record is processed)
  - E.g. column named *eventTime*

```
# In Python
from pyspark.sql.functions import *
(sensorReadings
  .groupBy("sensorId", window("eventTime", "5 minute"))
  .count())
```

Aggregations with Event-Time Windows

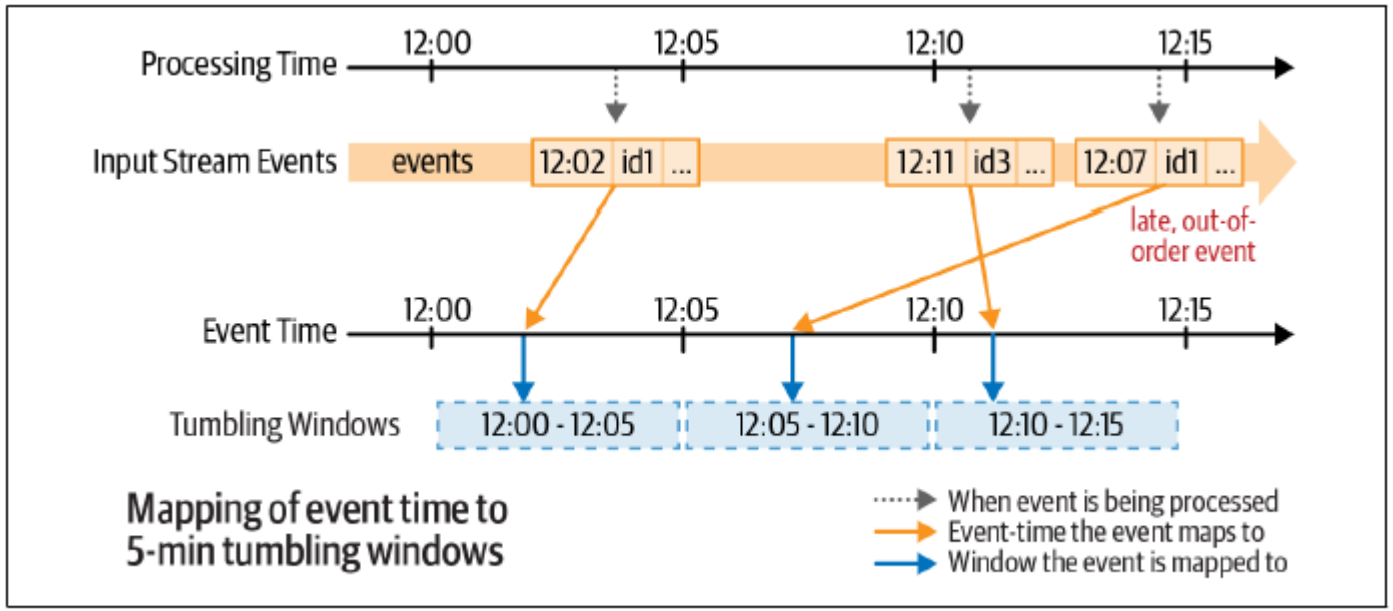


Figure 8-7. Mapping of event time to tumbling windows

```
# In Python
from pyspark.sql.functions import *
(sensorReadings
  .groupBy("sensorId", window("eventTime", "5 minute"))
  .count())
```

Aggregations with Event-Time Windows

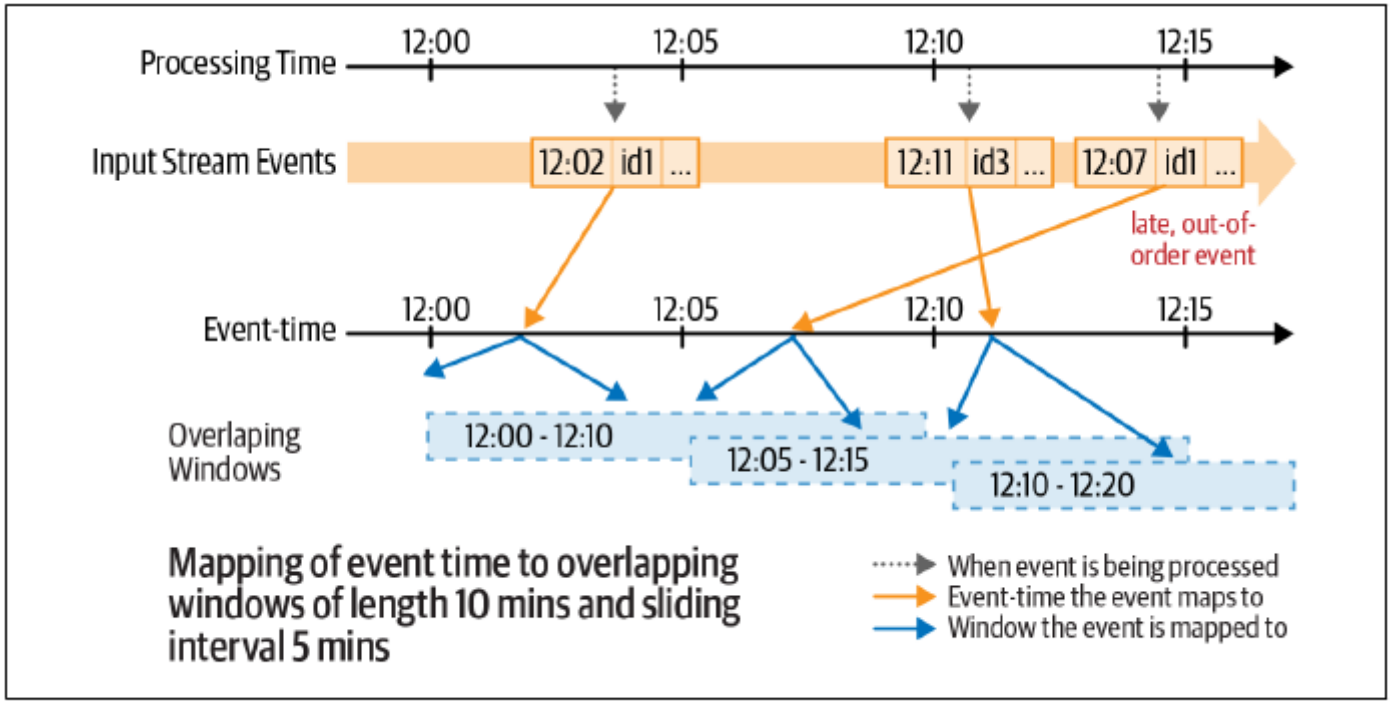


Figure 8-8. Mapping of event time to multiple overlapping windows

```
# In Python
(sensorReadings
  .groupBy("sensorId", window("eventTime", "10 minute", "5 minute"))
  .count())
```

Linked Data Analysis

Pregel: a system for large-scale graph processing, Malewicz, et al, *SIGMOD* 2010

Slides courtesy “Pregel: A System for Large-Scale Graph Processing, Malewicz, et al, SIGMOD 2010” and “Apache Giraph, Claudio Martella, Hadoop Summit, Amsterdam, April 2014”

Graphs are Commonplace

- Web & Social Networks
  - Web graph, Citation Networks, Twitter, Facebook, Internet
- Knowledge networks & relationships
  - Google's Knowledge Graph, NELL
- Cybersecurity
  - Telecom call logs, financial transactions, Malware
- Internet of Things
  - Transport, Power, Water networks
- Bioinformatics
  - Gene sequencing, Gene expression networks

Graphs are Commonplace

Smart Cities & IoT

Problem: Transport and Utility Network Analysis, Optimization

Challenges: data integration

Graph problems: Eulerian paths, MaxCut, Centrality

Cybersecurity

Problem: Detecting anomalies and bad actors

Challenges: scale, real-time

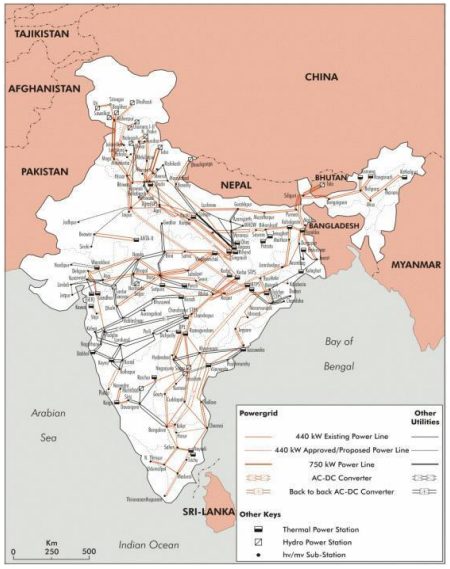
Graph problems: belief propagation, community analysis

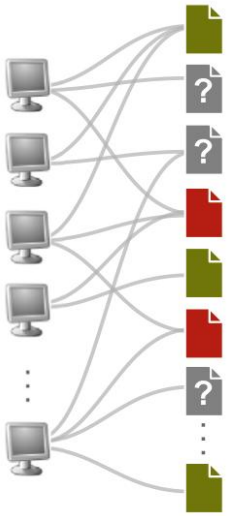
Social Informatics

Problem: Discover emergent communities, spread of info.

Challenges: new analytics routines, uncertainty in data.

Graph problems: clustering, shortest paths, flows.





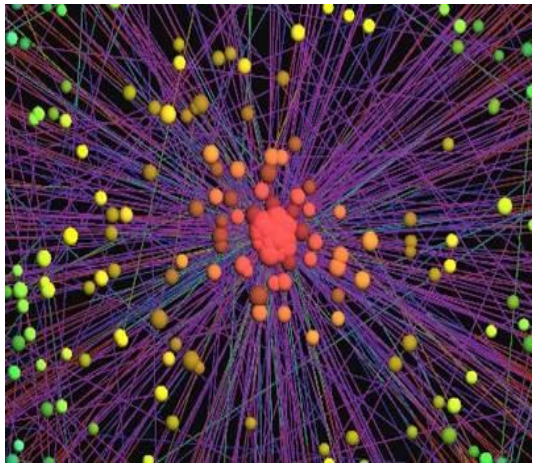


Image sources: (1) http://www.geni.org (2) Chau et al. In *SIAM Data Mining* (2011) (3) www.visualComplexity.com

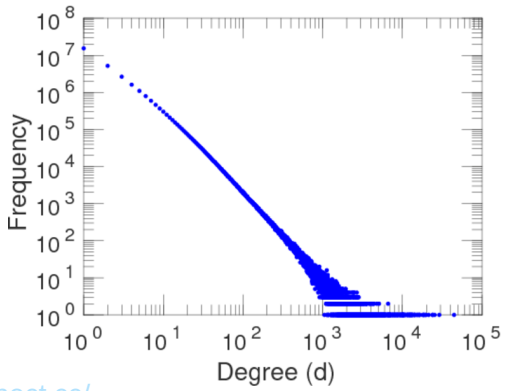
Graph Algorithms

- Traversals: *Paths & flows between different parts of the graph*
  - Breadth First Search, Shortest path, Minimum Spanning Tree, Eulerian paths, MaxCut
- Clustering: *Closeness between sets of vertices*
  - Community detection & evolution, Connected components, K-means clustering, Max Independent Set
- Centrality: *Relative importance of vertices*
  - PageRank, Betweenness Centrality

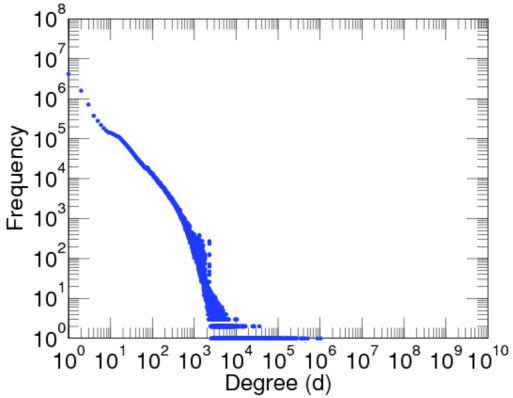
But, Graphs can be challenging

- Graphs are large in size…require distributed processing
- Graphs are irregular in structure
  - Adjacency matrix takes up a lot of space for sparse real-world graphs
  - Power-law distribution makes adjacency lists have highly distributed number of edges per vertex
  - Load on a worker is uneven when we partition a graph in a distributed system

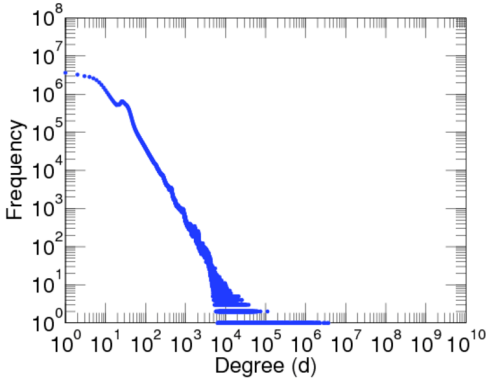
Amazon
V=31, E=82M



Wikipedia-EN
V=13M, E=437M



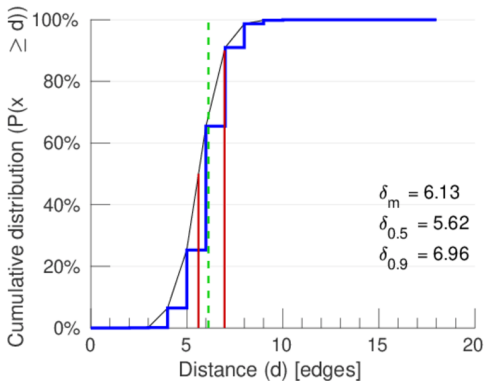
Twitter-MPI
V=52M, E=1,963M



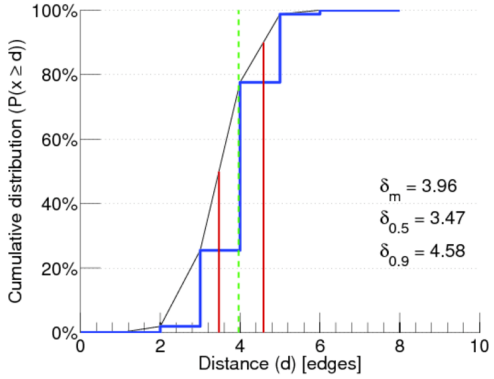
But, Graphs can be challenging

- Graph algorithms have uneven execution costs
  - E.g., BFS has a growing number of vertices as traversal depth increases

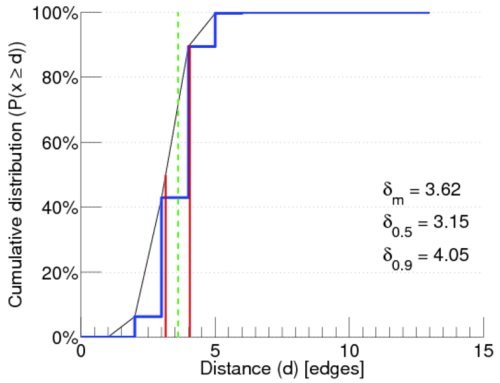
Amazon
V=31M, E=82M



Wikipedia-EN
V=13M, E=437M



Twitter-MPI
V=52M, E=1,963M



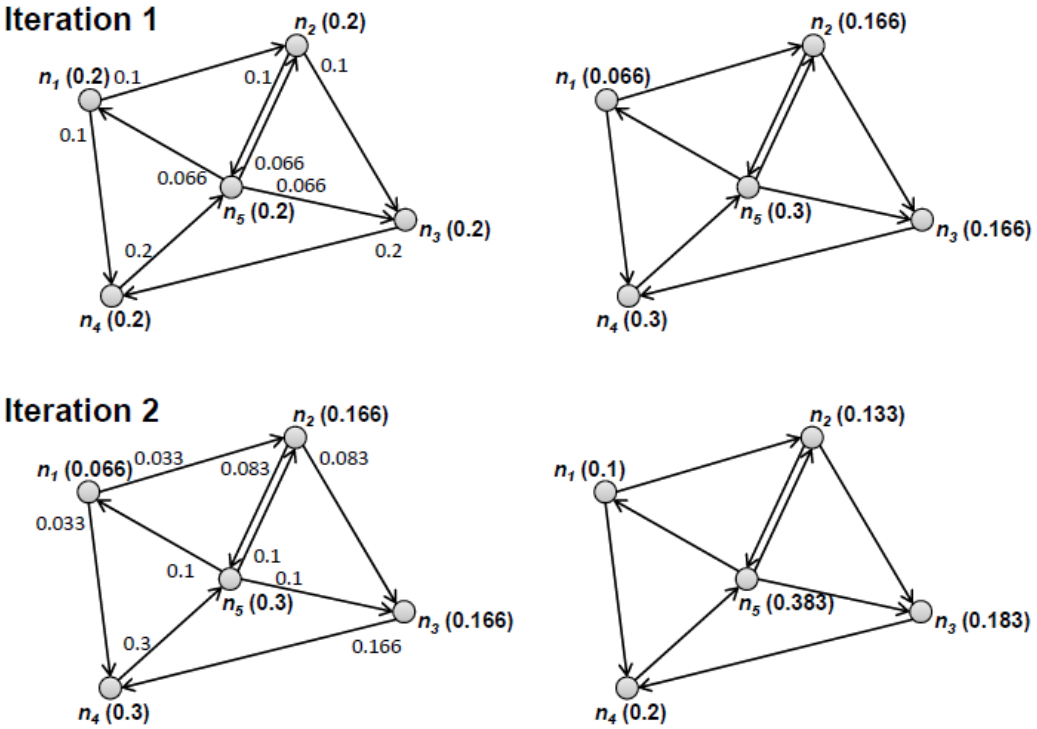
But, Graphs can be challenging

- Shared memory algorithms don't scale!
- Do not fit naturally to *MapReduce/Spark*
  - Multiple MR jobs (iterative MR)
  - *Tuple*, rather than graph-centric, abstraction
- Processing and *querying* are different
  - Graph DBs not suited for analytics
  - Focus on large simple graphs, complex "queries"
  - *E.g. Neo4J, FlockDB, 4Store, Titan*

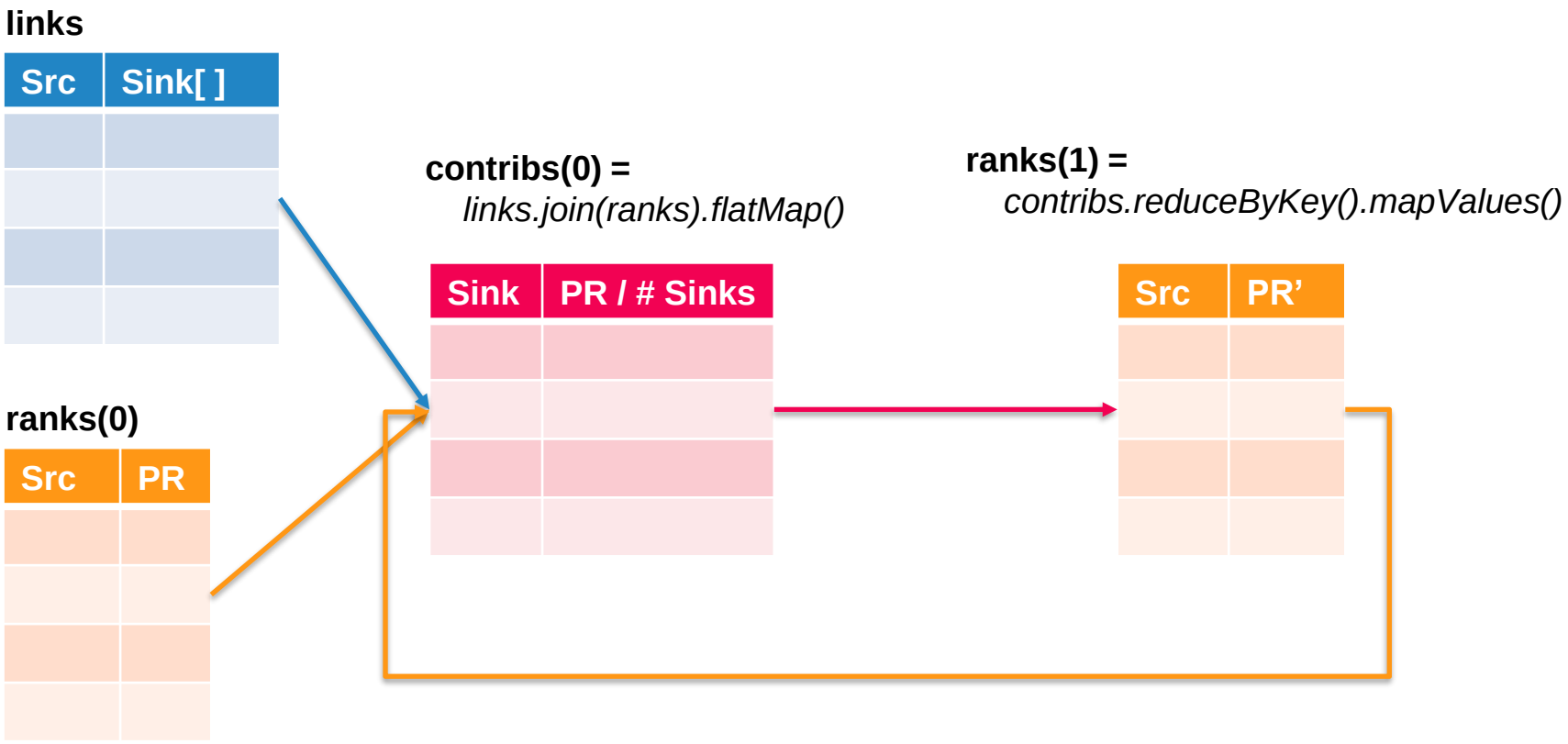
PageRank

$$P(n) = \alpha \left(\frac{1}{|G|} \right) + (1 - \alpha) \sum_{m \in L(n)} \frac{P(m)}{C(m)}$$

- Vertex Centrality metric. Importance of a vertex.
- Calculated iteratively
- Rank of vertex (**n**) depends on rank of neighbors (**L(n)**), normalized by # of out edges for neighbors (**C(m)**)



PageRank using Spark



PageRank using Spark

```
def computeContribs(urls, rank):
    for url in urls:
        yield (url, rank / len(urls))

# Loads all URLs and initialize their neighbors
links = lines.map(lambda urls: parseNeighbors(urls))
          .distinct().groupByKey().cache()

# Loads all URLs with other URL(s) link to from input file
# and initialize ranks of them to one
ranks = links.map(lambda url_neighbors: (url_neighbors[0], 1.0))

# Calculates and updates URL ranks continuously using PageRank algorithm.
for iteration in range(30):
    # Calculates URL contributions to the rank of other URLs.
    contribs = links.join(ranks).flatMap(lambda src_sinks_rank:
                  computeContribs(src_sinks_rank[1][0], src_sinks_rank[1][1]))

    # Re-calculates URL ranks based on neighbor contributions.
    ranks = contribs.reduceByKey(add).mapValues(lambda rank: rank*0.85 + 0.15)

# Collects all URL ranks and dump them to console.
for (link, rank) in ranks.collect():
    print("%s has rank: %s." % (link, rank))
```

https://github.com/apache/spark/blob/master/examples/src/main/python/pagerank.py

▷ *End of lecture*

PageRank using Spark: Challenges

- Not very intuitive
  - Graph algorithms tend to be **iterative** in nature. Causes repetitive joins.
  - Need to handle adjacency list, PR list, etc. separately
- Performance can be poor
  - Static graph structure needs to be shuffled repeatedly, once per iteration thru' **join**
  - Another shuffle per iteration to aggregate rank values using **reduceByKey**

Google's Pregel

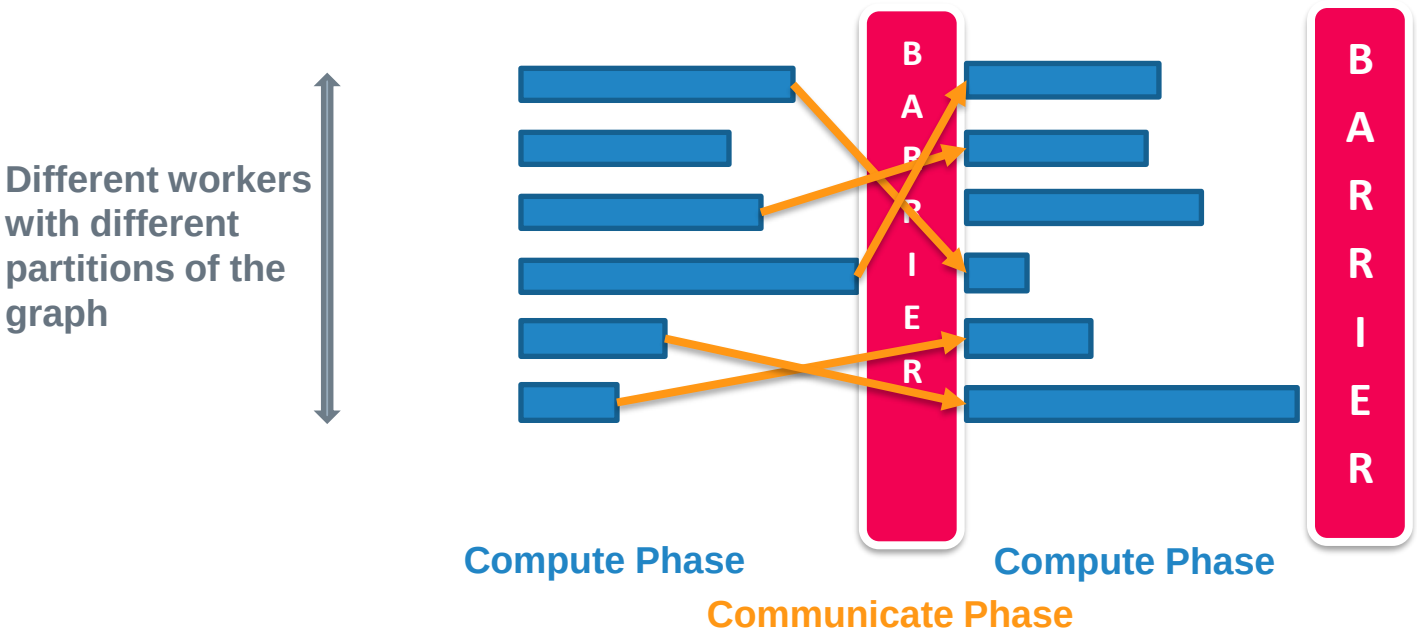
- Google proposed the Pregel programming model
  - Designed for iterative graph algorithms
  - Provides scalability & fault-tolerance
  - Flexibility to express arbitrary graph algorithms

- The high level organization of Pregel programs is inspired by Valiant's Bulk Synchronous Parallel (BSP) model [1].

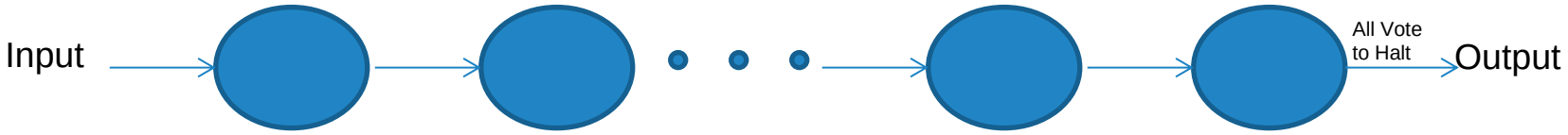
[1] Leslie G. Valiant, A Bridging Model for Parallel Computation. Comm. ACM 33(8), 1990

Bulk Synchronous Parallel (BSP)

- Distributed execution model
  - Compute → Communicate → Compute → Communicate → …
  - Bulk messaging avoids communication costs



Vertex-centric BSP



- Series of iterations (*supersteps*) .
- Each vertex *V* invokes a function in parallel.
- Can read messages sent in previous superstep (*S-1*).
- Can send messages, to be read at the next superstep (*S+1*).
- Can modify state of outgoing edges.

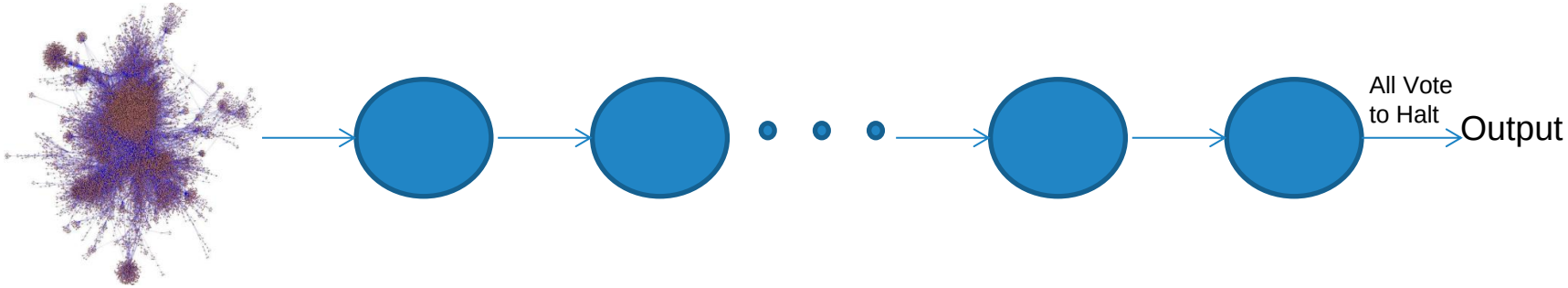
Advantage?

- In Vertex-Centric Approach
- Users focus on a *local action*
  - Think of **Map** method over tuple
- Processing each item *independently*.
- Ensures that Pregel programs are inherently free of *deadlocks* and *data races* common in asynchronous systems.

Apache Giraph
Implements *Pregel* Abstraction

- Google's Pregel, SIGMOD 2010
  - Vertex-centric Model
  - Iterative BSP computation
- Apache Giraph donated by Yahoo
  - Feb 6, 2012: Giraph 0.1-incubation
  - May 6, 2013: Giraph 1.0.0
  - Nov 19, 2014: Giraph 1.1.0
  - Oct 20, 2016: Giraph 1.2.0
  - Jun 11, 2020: Giraph 1.3.0
- Built on Hadoop Ecosystem

Model of Computation



- A Directed Graph is given to Pregel.
- It runs the computation at each vertex.
- Until all nodes vote for halt.
- Pregel gives you a directed graph back.

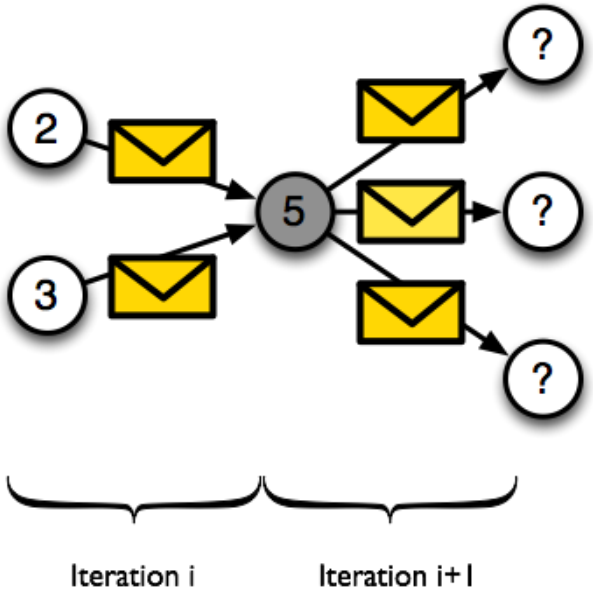
Vertex State Machine

- Algorithm termination is based on every vertex *voting to halt*.
- In superstep 0, every vertex is in the *active* state.
- A vertex deactivates itself by voting to halt.
- It can be reactivated by receiving an (external) message.

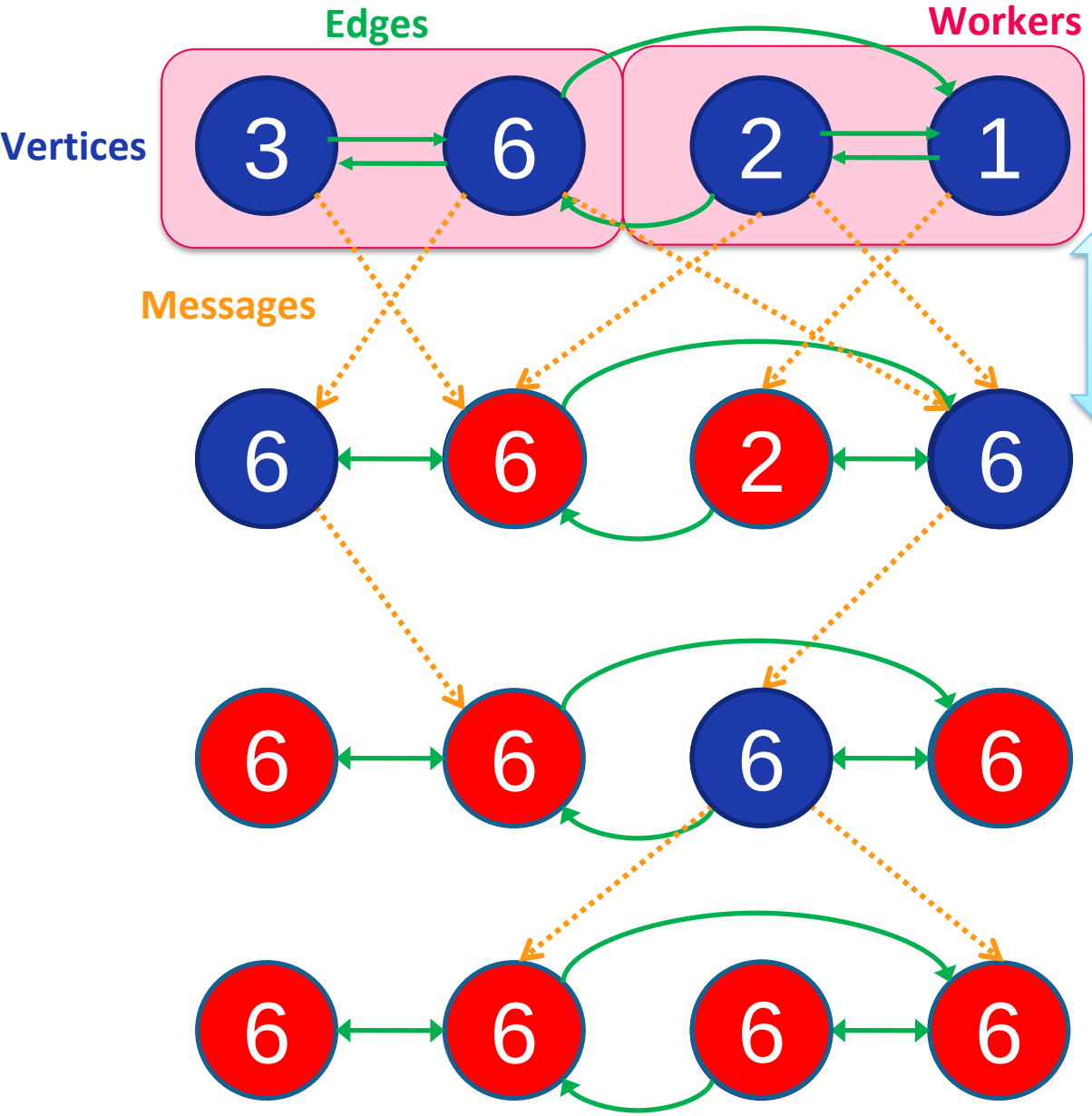
Vertex Centric Programming

- Vertex Centric Programming Model
  - Logic written from perspective on a single vertex. Executed on all vertices.

- Vertices know about
  - Their own value(s)
  - Their outgoing edges



Max Vertex Algo.



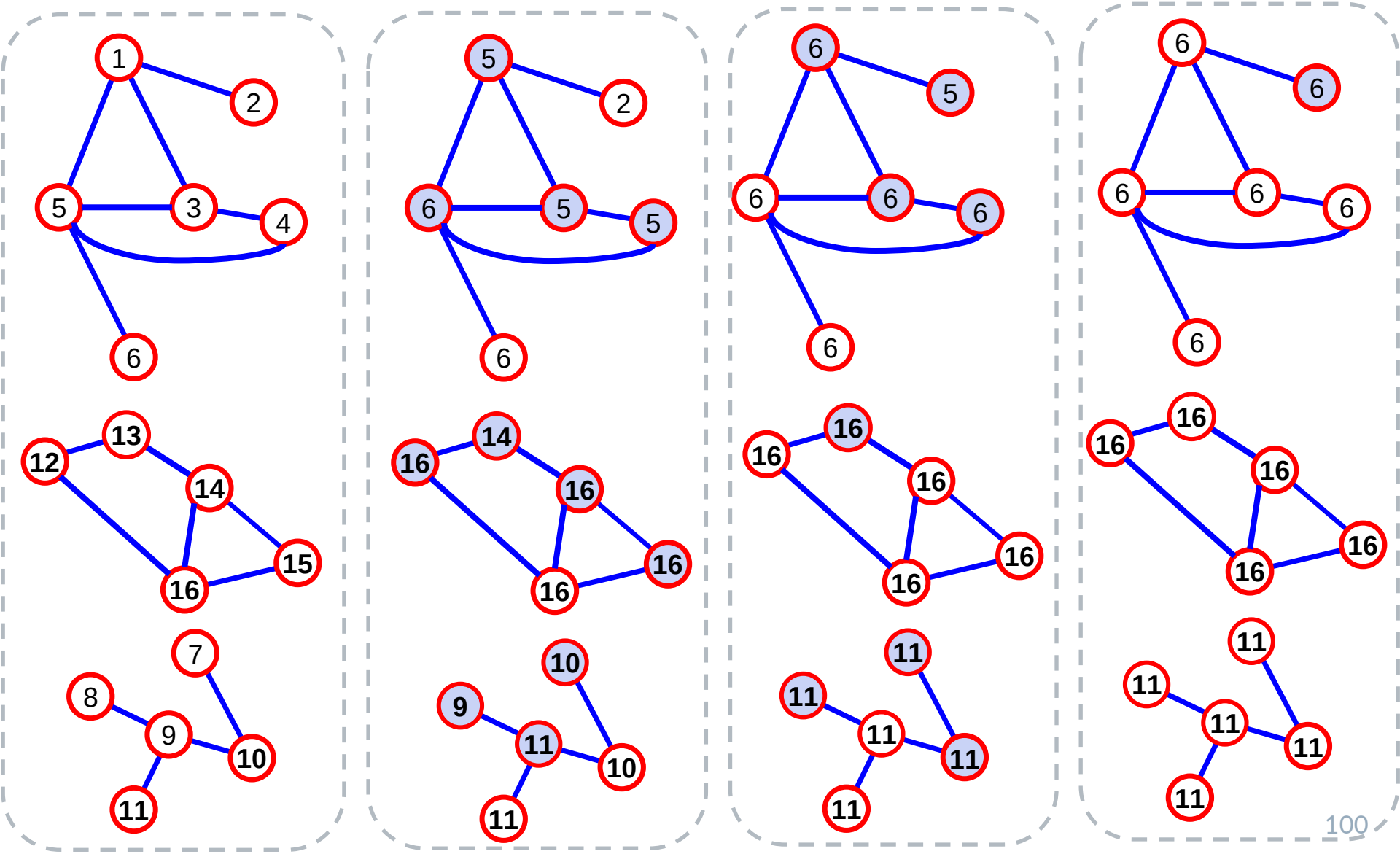
Blue Arrows are messages.

Blue vertices have voted to halt.

Max Vertex/ Weakly Connected Component (WCC)

```
void compute(Iterator<IntWritable> msgs) {
  hasChanged = (getSuperstep() == 0)
  for(m in msgs) {
    if(m > getVertexValue()) {
      setVertexValue(m)
      hasChanged = true
    }
  }
  if(hasChanged)
    sendMsgToAllEdges(getVertexValue())
  voteToHalt()
}
```

WCC

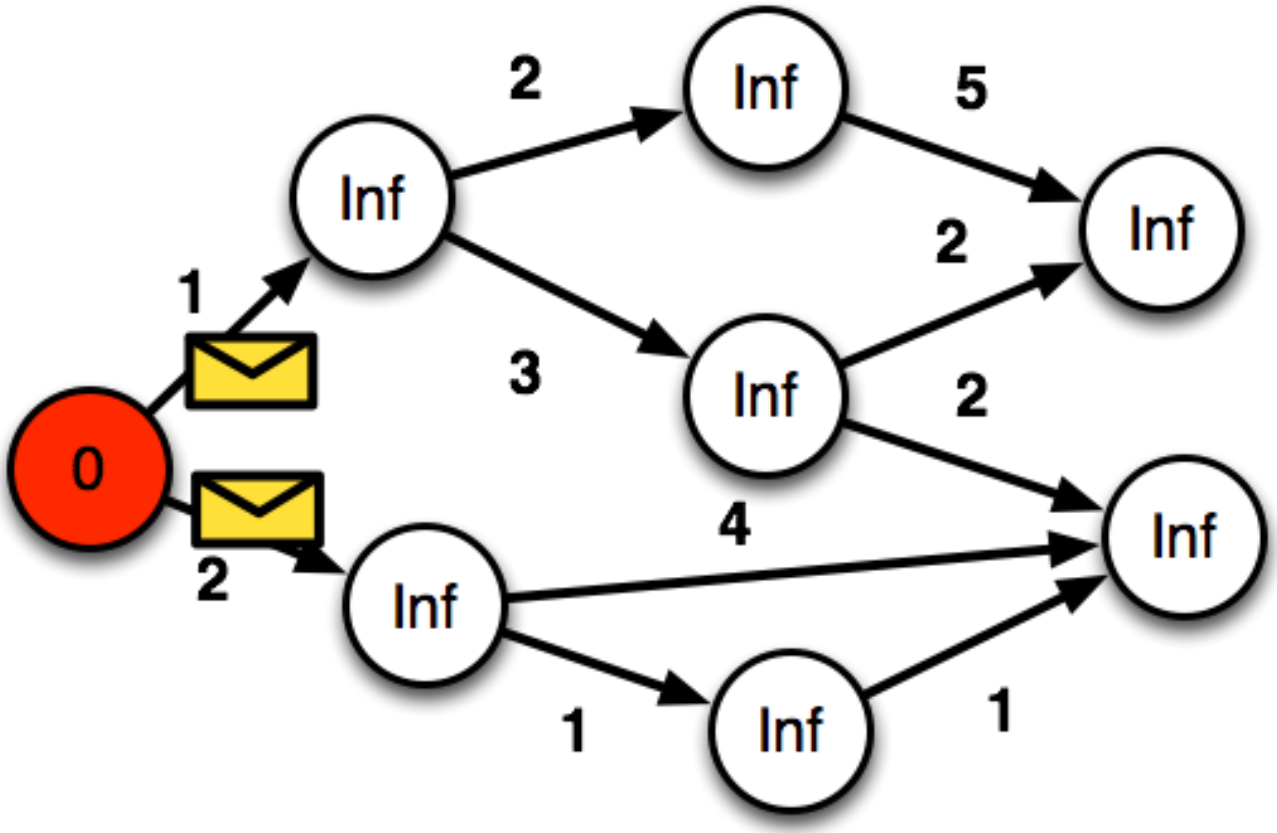

1
2
5
3
4
6
13
12
14
15
16
7
8
9
10
11
5
2
6
5
5
6
6
16
14
16
16
16
10
9
11
10
11
6
5
6
6
6
6
16
16
16
16
16
11
11
11
11
11
6
6
6
6
6
6
16
16
16
16
16
11
11
11
11
11

Shortest Path

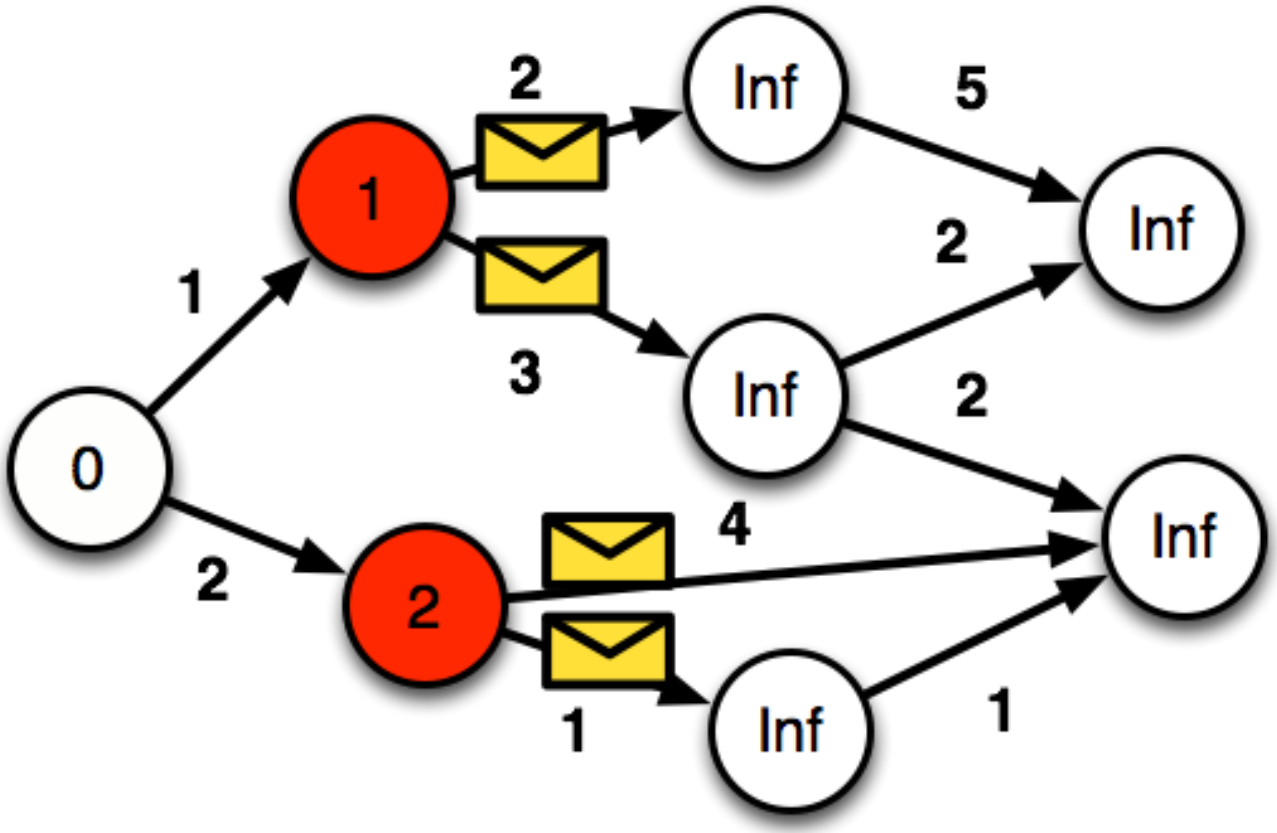
In the 1st superstep, only the source vertex will update its value (from INF to zero)

```
void compute(Iterator<IntWritable> msgs) {
  if(getSuperstep() == 0)
    setVertexValue(isSource(getVertexID())? 0 : INF)
  minDist = getVertexValue()
  for(m in msgs) {
    minDist = MIN(minDist, m)
  }
  if(minDist < getVertexValue()) {
    setVertexValue(minDist)
    for(e in edges())
      sendMsg(e.sink(), minDist + e.getValue())
  }
  voteToHalt()
}
```

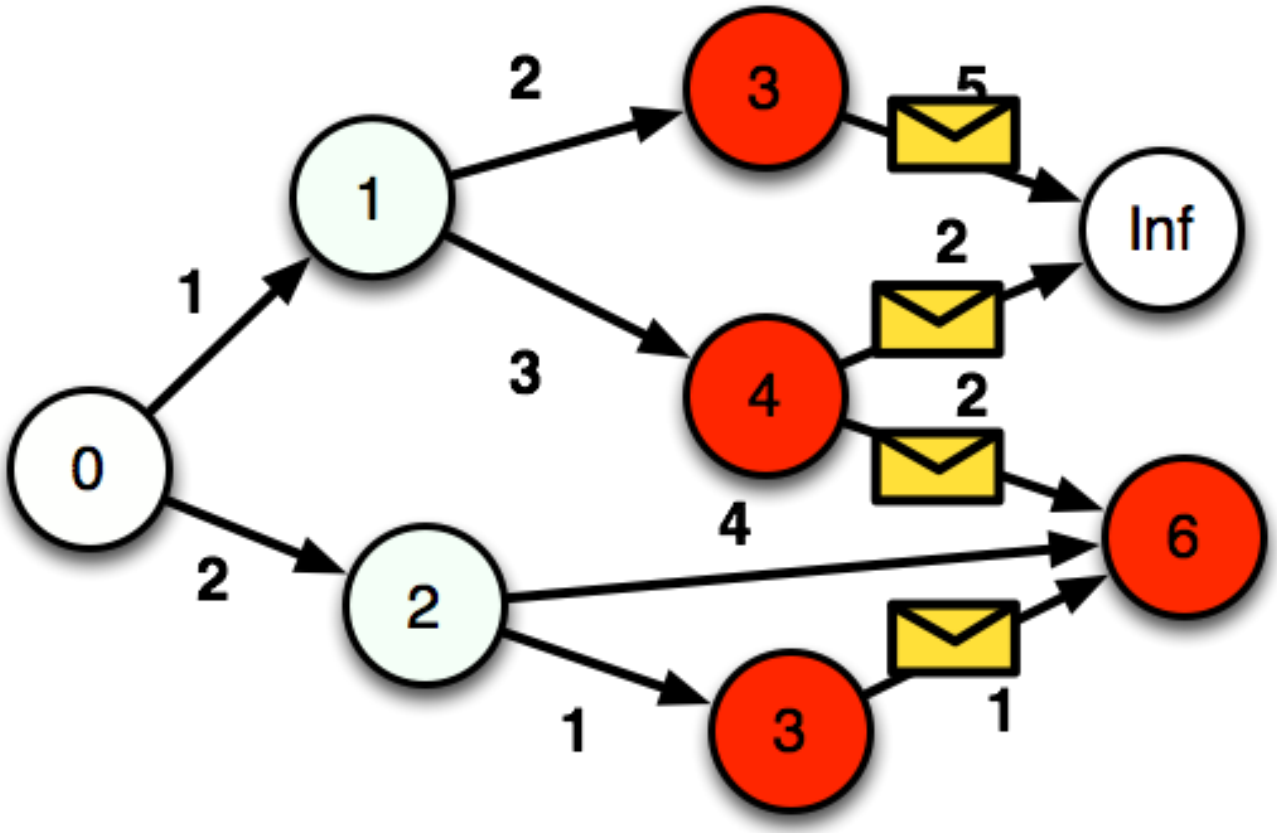
Shortest Path



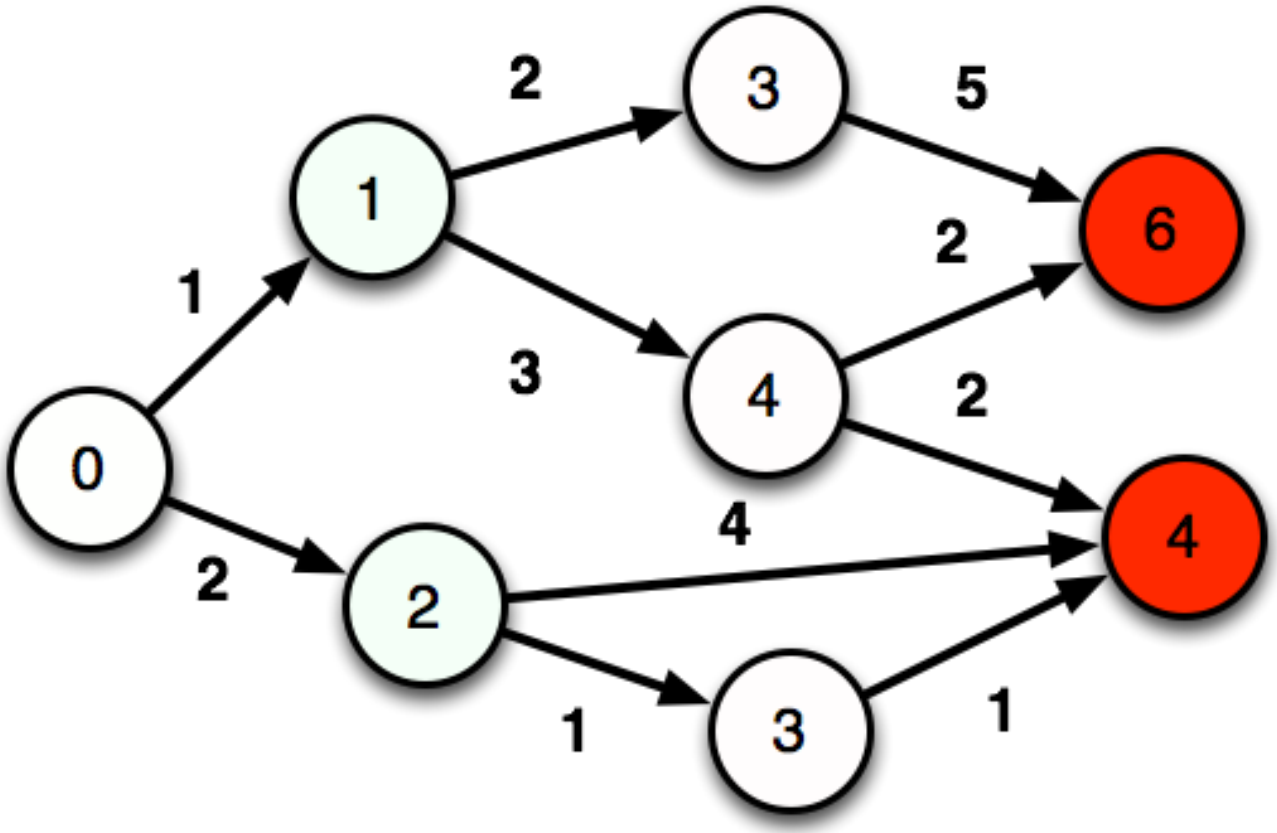
Shortest Path



Shortest Path



Shortest Path



PageRank, recursively

- P(n) is PageRank for webpage/URL ‘n’
  - Probability that you’re in vertex ‘n’
- |G| is number of URLs (vertices) in graph
- α is probability of random jump
- L(n) is set of vertices that link to ‘n’
- C(m) is out-degree of ‘m’

$$P(n) = \alpha \left(\frac{1}{|G|} \right) + (1 - \alpha) \sum_{m \in L(n)} \frac{P(m)}{C(m)}$$

PageRank

```
void compute(Iterator<IntWritable> msgs) {
  if(getSuperstep() == 0) setVertexValue(1/getVertexCount())
  if(getSuperstep() > 0) {
    for(m in msgs) {  sum += m  }
    // Or use alpha * sum + (1-alpha) * 1/getVertexCount()
    setVertexValue(sum)
  }
  if(getSuperstep() < 30) {
    n = edges().count()
    sendMsgToAllEdges(getVertexValue() / n)
  } else
    voteToHalt()
}
```

Apache Giraph: API

```
void compute(Iterator<IntWritable> msgs)
     getSuperstep()
     getVertexValue()
     edges = iterator()
     sendMsg(sinkVtx, value)
     sendMsgToAllEdges(value)
     voteToHalt()
```

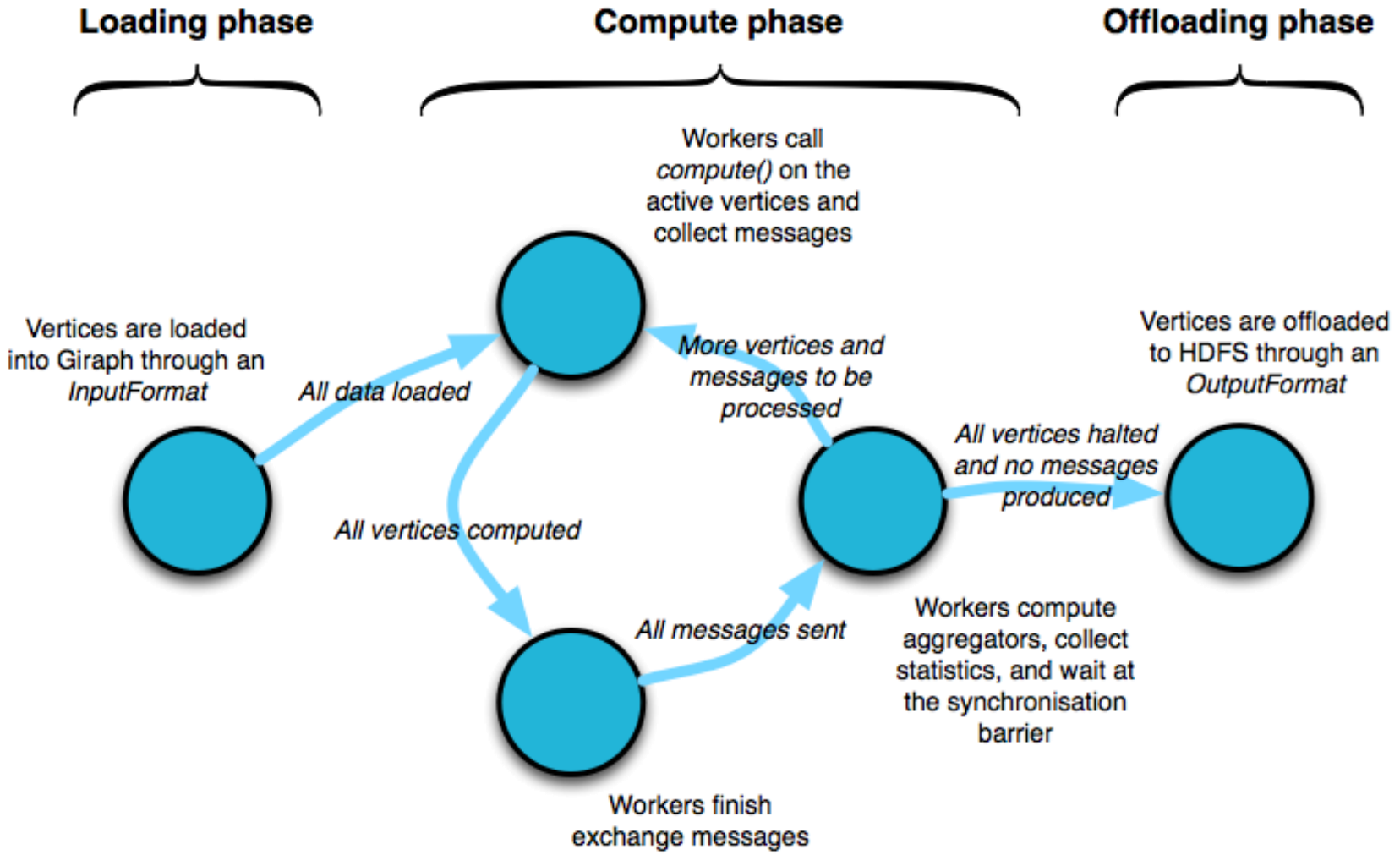
Advantages

- Makes distributed programming easy
  - No locks, semaphores, race conditions
  - Separates computing from communication phase
- Vertex-level parallelization
  - Bulk message passing for efficiency
- Stateful (in-memory)
  - Only messages & checkpoints hit disk

Message passing

- No guaranteed message delivery order.
- Messages are delivered exactly once.
- Can send messages to any node.
  - Though, typically to neighbors

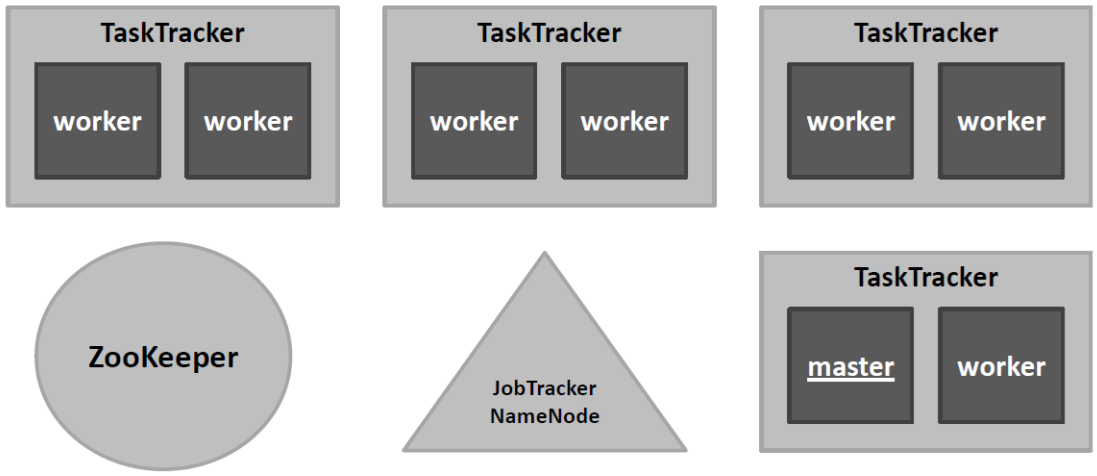
Apache Giraph



Apache Giraph, Claudio Martella, Hadoop Summit, Amsterdam, April 2014

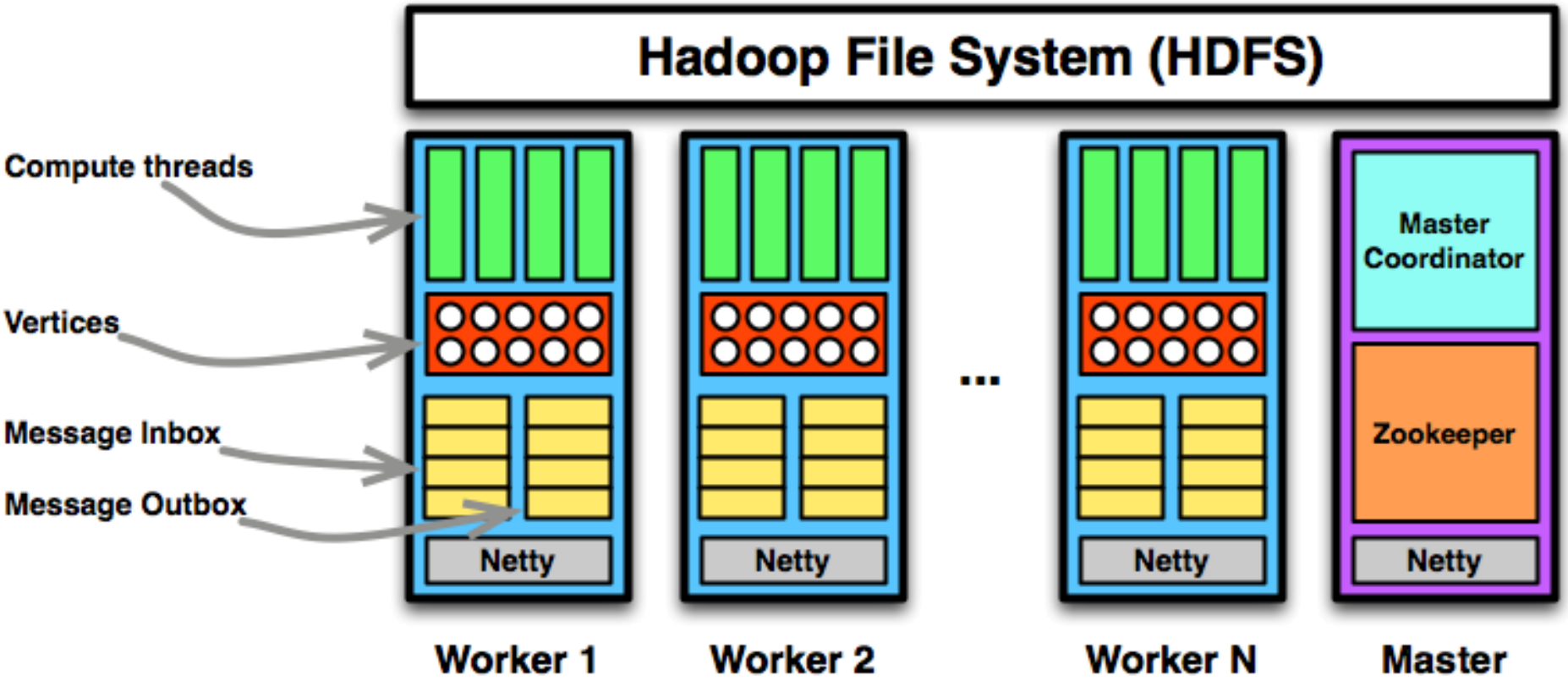
Giraph Architecture

- Hadoop Map-only Application
- **ZooKeeper**: responsible for **computation state**
  - Partition/worker mapping, global #superstep
- **Master**: responsible for **coordination**
  - Assigns partitions to workers, synchronization
- **Worker:** responsible for **vertices**
  - Invokes active vertices compute() function, sends, receives and assigns messages



Giraph Architecture

- Checkpointing of supersteps possible



Partitioner

- Maps vertices to partitions that are operated by workers
  - Default is a hash partitioner
- Done once at the start of the application
- Called at the end of each superstep, for dynamic migration of partitions

Combiners

- Sending a message to remote vertex has overhead
  - Can we merge multiple *incoming* message into one?
- User specifies a way to reduce many messages into one value (ala Reduce in MR)
  - by overriding the **Combine()** method.
  - Must be commutative and associative.

```
originalMessage =
 combine(vid, originalMessage, messageToCombine)
```

- Exceedingly useful in certain contexts (e.g., 4x speedup on shortest-path computation).
  - e.g. for MAX, om = om < mtc ? mtc : om

End of Lecture/Module/Course

Additional Slides

Not for Quizzes/Exams

K-Means Clustering

```
1  Input: undirected G(V, E), k, τ
2   int  numEdgesCrossing = INF;
3   while (numEdgesCrossing > τ)
4     int []  clusterCenters  = pickKRandomClusterCenters(G)
5     assignEachVertexToClosestClusterCenter(G, clusterCenters)
6     numEdgesCrossing = countNumEdgesCrossingClusters(G)
```

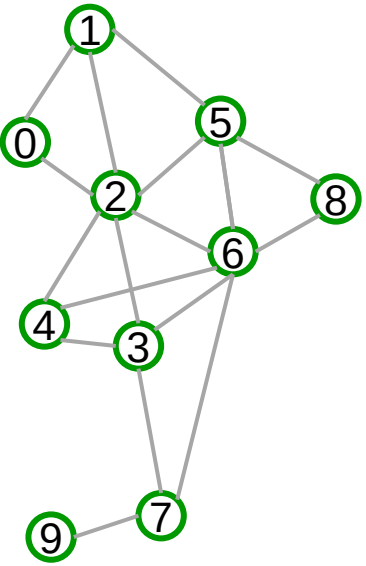
Figure 3: A simple k-means like graph clustering algorithm.

```
1  public class  EdgeCountingVertex extends
2          Vertex<IntWritable, IntWritable> {
3    @Override
4    public void compute(Iterable<IntWritable> messages,
5                        int  superstepNo){
6      if  (superstepNo == 1) {
7        sendMessages(getNeighborIds(), getValue().value());
8      } else  if  (superstepNo == 2) {
9        for  (IntWritable message : messages) {
10         if  (message.value() != getValue().value())  {
11           minValue = message.value();
12           updateGlobalObject(''num−edges−crossing−clusters'',
13             new IntWritable(1));}}
14       voteToHalt(); }}}
```

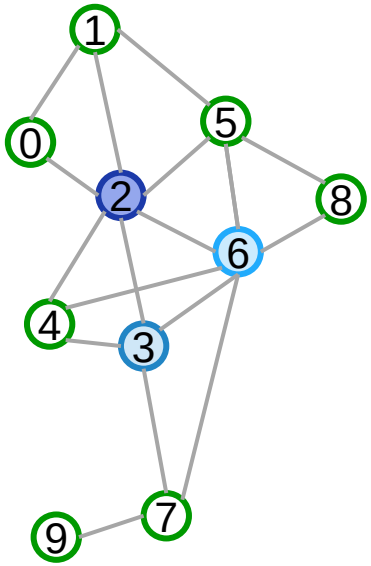
Figure 4: Counting the number of edges crossing clusters with *vertex.compute()*.

- Multiple phases
  - k centers
  - assign vertex to cluster
  - find edge cuts
- Use multi-source BFS or Euclidian distance to find nearest cluster
- Use MasterCompute
  - k initial vertices
  - Calc edge-cut count
  - Decide termination

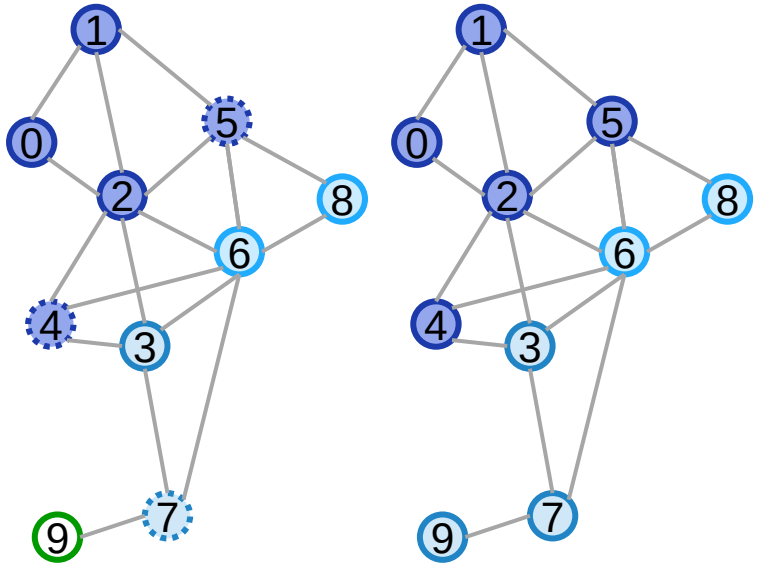
GPS: A Graph Processing System, Semih Salihoglu et al, SSDBM 2013
One Trillion Edges: Graph Processing at FacebookScale, Avery Ching, et al, VLDB 2015



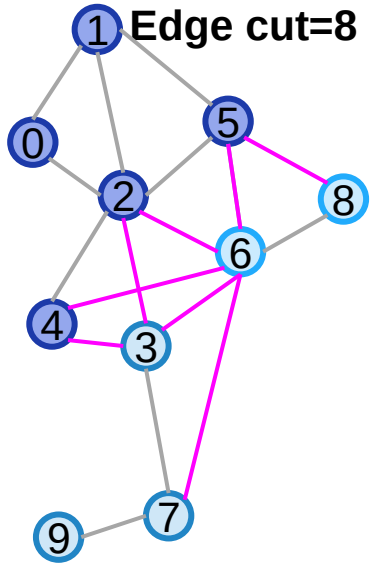
Load



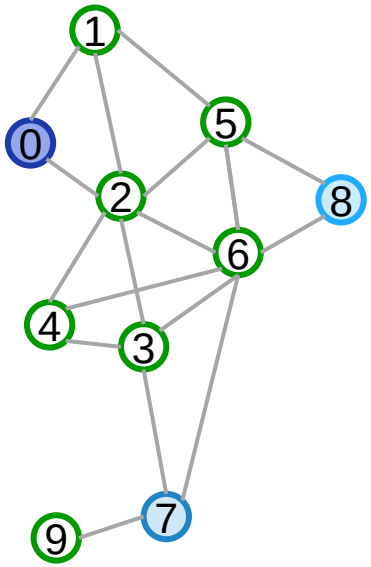
Select Centroid

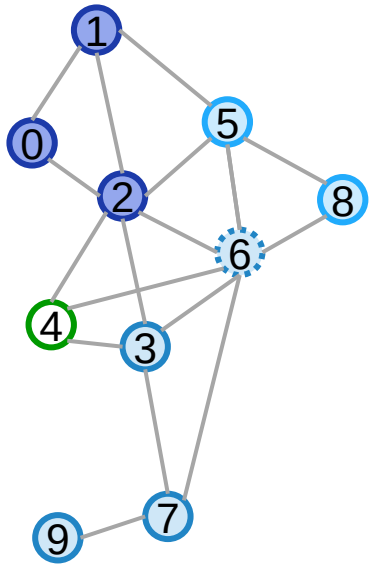


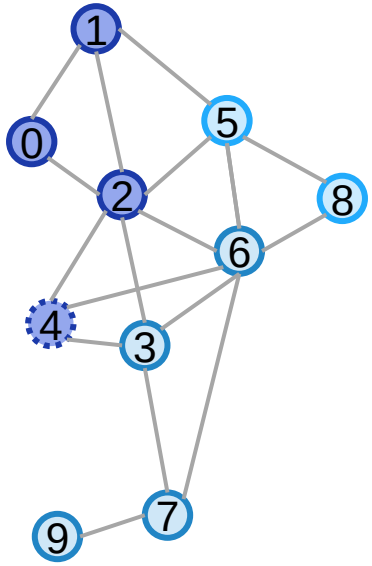
Label Propagation

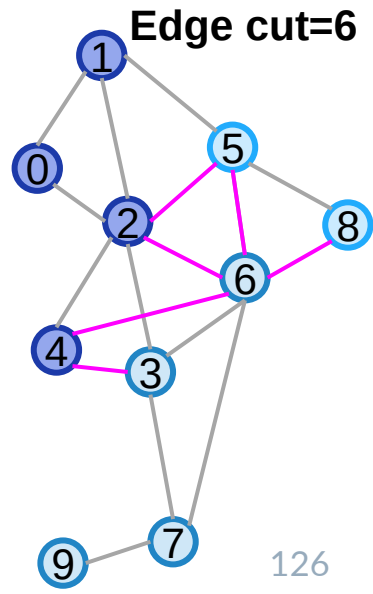


Find Edge Cut









Breadth First Search (BFS)

```
void compute(Iterator<IntWritable> msgs) {
  if(getSuperstep() == 0)
    setVertexValue(isSource(getVertexID())? 0 : INF)

  if(getVertexValue() == INF) {
    // Only first visit matters.
    // m.count > 0. All messages have same value
    setVertexValue(m.first()) // getSuperstep
                              works too
    sendMsgToAllEdges(m.first() + 1)
  }
  voteToHalt()
}
```

BFS